UNIVERSITAS INDONESIA

LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

SKRIPSI

MUHAMAD ABDURAHMAN
1706040095

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JULI 2021

UNIVERSITAS INDONESIA

LEX2KG: AUTOMASI PEMBUATAN KNOWLEDGE GRAPH DARI DOKUMEN PERATURAN PERUNDANG-UNDANGAN INDONESIA

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Ilmu Komputer

MUHAMAD ABDURAHMAN
1706040095

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JULI 2021

HALAMAN PERSETUJUAN

Judul	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia
Nama	:	Muhamad Abdurahman
NPM	:	1706040095

Laporan Skripsi ini telah diperiksa dan disetujui.

8 Juli 2021

Fariz Darari, Ph.D.

Pembimbing Skripsi

HALAMAN PERNYATAAN ORISINALITAS

Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.

Nama	:	**Muhamad Abdurahman**
NPM	:	**1706040095**
Tanda Tangan	:	
Tanggal	:	**8 Juli 2021**

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh	:	
Nama	:	Muhamad Abdurahman
NPM	:	1706040095
Program Studi	:	Ilmu Komputer
Judul Skripsi	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Ilmu Komputer pada Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing 1	:	Fariz Darari, Ph.D.	(	)
Penguji 1	:		(	)
Penguji 2	:		(	)

Ditetapkan di	:	Depok
Tanggal	:	

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama	:	Muhamad Abdurahman
NPM	:	1706040095
Program Studi	:	Ilmu Komputer
Fakultas	:	Ilmu Komputer
Jenis Karya	:	Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif (Non-exclusive Royalty Free Right)** atas karya ilmiah saya yang berjudul:

Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (database), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyatan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : 8 Juli 2021

Yang menyatakan

(Muhamad Abdurahman)

KATA PENGANTAR

Segala puji serta syukur dipanjatkan kepada Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas akhir ini. Tugas akhir ini dibuat sebagai wujud kontribusi penulis pada bidang penelitian penulis, almamater Universitas Indonesia, dan juga untuk membantu perkembangan teknologi di Indonesia. Penulis menyadari bahwa banyak sesi bimbingan, arahan, dukungan moral, bantuan teknis dan doa dari banyak pihak yang membantu dalam menyelesaikan tugas akhir ini. Oleh karena itu, penulis ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Ibu Syifa Felicia dan Bapak Miftahul Huda, selaku orang tua yang penulis cintai. Terima kasih atas segala dukungan, doa, dorongan, semangat, dan kasih sayang yang telah diberikan sepanjang hidup penulis.

2. Bapak Fariz Darari, Ph.D. selaku dosen pembimbing penulis selama pengerjaan tugas akhir ini. Terima kasih atas segala pengetahuan, bimbingan, saran, waktu, dan tenaga yang telah Bapak berikan selama membimbing penulis.

3. Bapak Ir.Suryana Setiawan Ph.D. selaku pembimbing akademis selama saya berkuliah di Fasilkom.

4. Teman-teman server discord "Bambang/Misan" yang menemani saya secara daring selama pandemi.

5. Semua pihak lain yang telah membantu penulis yang tidak dapat disebutkan satu per satu.

Depok, 8 Juli 2021

Muhamad Abdurahman

ABSTRAK

Nama	:	Muhamad Abdurahman
Program Studi	:	Ilmu Komputer
Judul	:	Lex2KG: Automasi Pembuatan Knowledge Graph dari Dokumen Peraturan Perundang-undangan Indonesia

Dokumen peraturan perundang-undangan pada umumnya tersedia dalam bentuk PDF yang bersifat tidak *machine-readable*. *Knowledge graph* (KG) adalah *graph* yang menggambarkan entitas dunia nyata beserta keterkaitannya, memberikan informasi terstruktur yang *machine-readable*. Pada penelitian penulis mengembangkan Lex2KG, *framework* untuk mengonversi dokumen peraturan perundang-undangan (PDF) di Indonesia menjadi KG. KG peraturan perundang-undangan mengandung berbagai data terstruktur konten tekstual, struktur dokumen, seperti metadata, serta relasi antara peraturan seperti amendemen dan rujukan. Menggunakan Lex2KG, penulis berhasil mengonversi 784 undang-undang menjadi KG dengan ukuran total lebih dari 1,1 juta *triple*. Salah satu peraturan yang berhasil dikonversi adalah UU 11/2020 tentang Cipta Kerja yang kontennya bersifat relatif kompleks dan berukuran besar. Penulis juga menunjukan *use case* dari KG peraturan perundang-undagan yaitu *chat bot* sederhana, SPARQL *query*, dan visualisasi peraturan perundang-undangan.

Kata kunci:
Knowledge Graph, Peraturan perundang-undangan, SPARQL, RDF, Konversi, PDF

ABSTRACT

Name : Muhamad Abdurahman
Program : Ilmu Komputer
Title : Lex2KG: Automated Legal KG Population from Indonesian Legal Documents

Most of the legal documents are available as PDF which is not machine-readable. A knowledge graph (KG) is a graph that describes real-world entities and their relationships as machine-readable and structured information. In this research, author developed Lex2KG, a framework for converting Indonesian legal (PDF) documents into a KG. The legal KG contains various kinds of structured data such as textual content, document structures, metadata, and relations between law such as amendments and citations. Through Lex2KG, author have successfully converted 784 Indonesian laws into a KG with a total size of over 1.1 million triples. One of the regulation that was successfully converted was Law 11/2020 on Job Creation, which the content is relatively complex and large. Author also shows use cases of the legal KG for simple chatbots, SPARQL querying, and legal visualizations.

Key words:
Knowledge Graph, Law, SPARQL, RDF, Conversion, PDF

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR KODE PROGRAM

DAFTAR LAMPIRAN

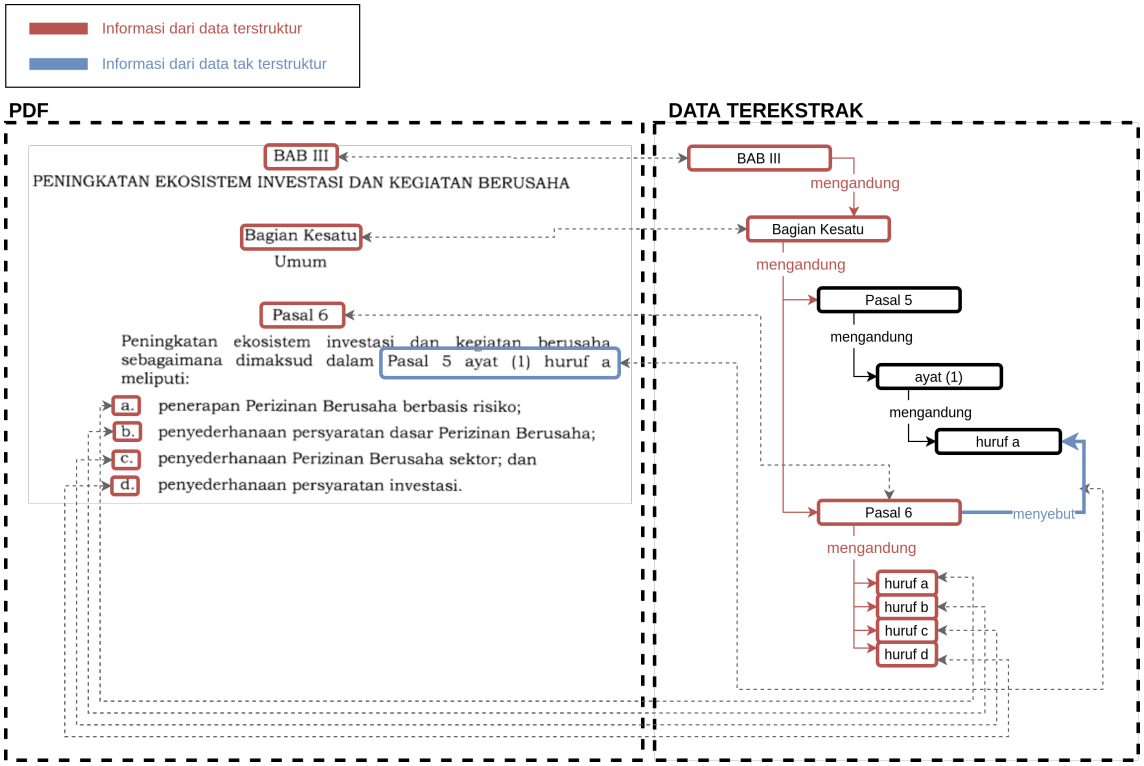
BAB 1
PENDAHULUAN

1.1 Latar Belakang

Peraturan perundang-undangan adalah peraturan tertulis yang memuat norma hukum yang mengikat secara umum dan dibentuk atau ditetapkan oleh lembaga negara atau pejabat yang berwenang melalui prosedur yang ditetapkan dalam peraturan perundang-undangan, sesuai yang dijelaskan dalam Undang-Undang Republik Indonesia Nomor 12 Tahun 2011 (selanjutnya disebut dengan UU 12/2011). Peraturan perundang-undangan dapat digunakan untuk menjawab pertanyaan-pertanyaan yang berkaitan dengan hukum, seperti:

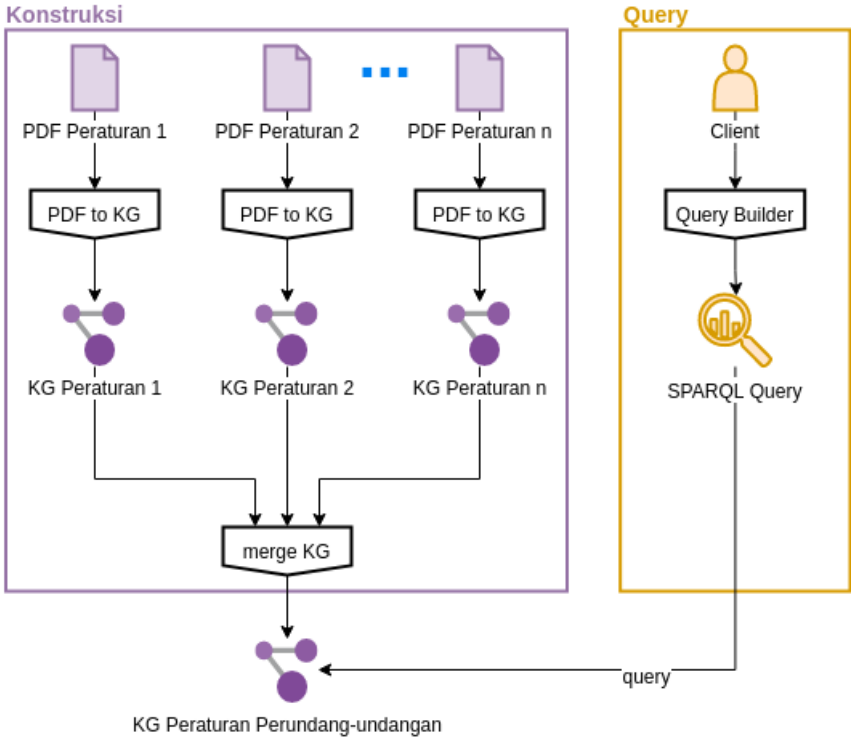
- Peraturan apa yang berlaku pada daerah X yang disahkan di tahun Y?
- Apa relasi suatu peraturan dengan peraturan lain?
- Mana saja peraturan yang membahas tentang topik W atau X tapi bukan topik Y dan bukan Z?
- Apa isi dari Pasal X ayat (Y) huruf Z dari suatu peraturan perundang-undangan?
- Apa saja bab-bab yang terdapat pada peraturan X?

Pertanyaan-pertanyaan tersebut umumnya biasa dijawab oleh seorang ahli hukum, yang hanya dapat dilakukan dalam skala kecil dan biaya relatif mahal. Komputer dapat menjadi alternatif untuk aplikasi tersebut dalam skala lebih besar dan biaya lebih murah, jika peraturan perundang-undangan berupa data terstruktur yang dapat diolah oleh komputer. Saat ini pembuatan dan pemanfaatan peraturan perundang-undangan masih dilakukan secara manual oleh manusia. Pembuatan peraturan perundang-undangan dilakukan dengan mengetikkan konten peraturan tersebut dengan format yang telah ditentukan. Metode pembuatan tersebut tidak masalah jika hanya akan dibaca oleh manusia karena manusia secara tidak sadar dapat melihat hirarki visual dari dokumen tersebut dan membuat data yang dilihatnya terstruktur di dalam otak. Metode ini membuat pemanfaatan peraturan perundang-undangan oleh mesin menjadi kurang efisien karena mesin perlu melakukan proses tambahan yaitu mengonversi gambar menjadi data terstruktur.



Gambar 1.1: Cuplikan ekstraksi data terstruktur dan data tidak terstruktur dari dokumen PDF UU 11/2020 tentang Cipta Kerja

Gambar **??** menampilkan contoh ekstraksi informasi dari UU 11/2020 tentang Cipta Kerja. Pada gambar tersebut terlihat bahwa terdapat dua jenis ekstraksi data yaitu ekstraksi dari data terstruktur (merah) dan ekstraksi dari data tidak terstruktur (biru). Contoh data terstruktur pada dokumen tersebut adalah *section* pada dokumen seperti **Bab III**, **Bagian Kesatu**, **Pasal 6**, dan **huruf a - d**. Contoh data tidak terstruktur pada dokumen tersebut adalah semua teks yang terdapat pada *section* tersebut seperti teks "penerapan Perizinan Berusaha berbasis risiko;" yang terdapat pada **huruf a**. Manusia perlu membaca manual untuk mengetahui bahwa UU 11/2020 mengandung **Bab III**, **Bab III** mengandung **Bagian Kesatu**, **Bagian Kesatu** mengandung **Pasal 6**, dan **Pasal 6** mengandung **huruf a - d**. Pada gambar tersebut, dalam mengekstraksi informasi "UU 11/2020 Pasal 6 merujuk kepada Pasal 5 ayat (1) huruf a", manusia pertama-tama perlu membaca dari awal teks pasal-pasal pada UU perujukan (citation), dan kemudian meng-*infer* informasi lengkap berdasarkan konteks peraturan perundang-undangan.



Gambar 1.2: Bagan alir pembuatan dan *query* KG peraturan perundang-undangan

Knowledge graph (KG) adalah struktur data yang (i) menggambarkan entitas dunia nyata dan keterkaitannya, tersusun dalam bentuk *graph*, (ii) mendefinisikan kelas entitas dan relasi antar entitas dalam bentuk skema, (iii) membolehkan sembarang entitas memiliki relasi satu sama lain dan (iv) mencakup berbagai topik domain (**?**). Dengan melakukan ekstraksi data dari dokumen yang bersifat semi-terstruktur, peraturan perundang-undangan dapat dikonversi menjadi KG. Sebagai manfaatnya, dapat fitur-fitur berbasis *query* seperti *question answering*, *reasoning* dan visualisasi data. Gambar **??** menampilkan proses konstruksi KG dari beberapa berkas PDF peraturan perundang-undangan dan *query* pada KG tersebut. Pada skripsi ini, akan dilakukan konversi dokumen peraturan perundang-undangan menjadi data terstruktur dalam bentuk KG, dan memberikan contoh aplikasi dari KG peraturan perundang-undangan tersebut. Beberapa usaha telah dilakukan untuk membuat *vocabulary* untuk KG peraturan perundang-undangan seperti ELI,[1] Schema.org,[2] dan Wikidata.[3]

[1] `https://eur-lex.europa.eu/eli-register/about.html`
[2] `https://schema.org`
[3] `https://www.wikidata.org`

```
@prefix schema: <http://schema.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<http://data.europa.eu/eli/dir/1980/181/oj> a schema:Legislation ;
  schema:about <http://eurovoc.europa.eu/1896> ;
  schema:isBasedOn <http://njh.me/11957E100.html> ;
  schema:legislationDate "1979-12-20"^^schema:Date ;
  schema:legislationPassedBy "Council of the European
   ↪ Union"^^xsd:string ;
  schema:name "Council Directive 80/181/EEC of 20 December 1979 on the
   ↪ approximation of the laws of the Member States relating to units
   ↪ of measurement and on the repeal of Directive
   ↪ 71/354/EEC"^^xsd:string ;

<http://eurovoc.europa.eu/1896> a schema:Thing ;
  schema:name "Metrology"^^xsd:string .

<http://njh.me/11957E100.html> a schema:Legislation .
```

Kode 1.1: Contoh KG metadata dokumen legal menggunakan *vocabulary* Schema.org

Pada Kode **??** diberikan contoh KG yang dibuat dengan *vocabulary* Schema.org. Contoh KG tersebut memuat metadata peraturan seperti tentang apa peraturan tersebut (`schema:about`), apa dasar dari peraturan tersebut (`schema:isBasedOn`), dan nama dari peraturan tersebut (`schema:name`). Akan tetapi, baik Schema.org maupun vocabulary-vocabulary lainnya seperti pada ELI dan Wikidata, tidak dapat merepresentasikan struktur beserta isi peraturan. Selain itu, *vocabulary* tersebut dirancang secara generik, sehingga tidak dapat merepresentasikan data yang secara spesifik terdapat pada peraturan perundang-undangan di Indonesia. Sehingga dengan membuat *vocabulary* untuk struktur data dan peraturan perundang-undangan di Indonesia, diharapkan dapat menjadi kontribusi dalam mendukung pembuatan KG untuk dokumen peraturan perundang-undangan.

Lex2KG merupakan *framework* yang dikembangkan pada penelitian ini, dapat mengonversi dokumen menjadi KG secara otomatis. Belum ada pekerjaan terkait yang melakukan automasi konversi dokumen peraturan perundang-undangan menjadi KG secara otomatis. Oleh karena itu Lex2KG diharapkan dapat menjadi kontribusi yang mendukung pembuatan dan penggunaan KG.

1.2 Permasalahan

Berikut adalah rumusan permasalahan dari penelitian yang dilakukan:

- Bagaimana merancang suatu sistem untuk melakukan konversi otomatis PDF peraturan perundang-undangan menjadi KG?
- Bagaimana membuat ontologi untuk memperjelas *requirement* dan mempermudah konversi dokumen PDF menjadi KG?
- Bagaimana evaluasi use case terhadap kg peraturan perundang-undangan tersebut serta evaluasi konversi dalam skala besar?

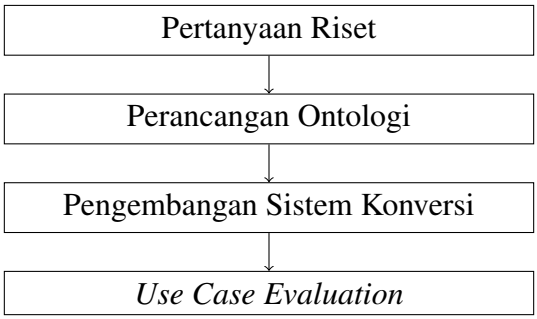
1.3 Batasan Permasalahan

Berikut adalah batasan dari penelitian yang dilakukan:

- Konversi hanya dapat dilakukan pada peraturan perundang-undangan di Indonesia
- Peraturan yang dipelihara kualitas konversinya dibatasi kepada beberapa peraturan UU dan non-UU karena evaluasi kualitas konversi harus dilakukan secara manual
- Peraturan yang dipelihara kualitas konversinya memiliki jumlah yang *imbalanced* untuk UU dan peraturan jenis lainnya
- Tidak dapat melakukan *query* terhadap KG yang kompleks seperti *query* yang membutuhkan *natural language processing* (contoh: "Jika saya berusia 12 tahun, apakah saya boleh bekerja menurut UU Ketenagakerjaan?")

1.4 Metodologi Penelitian

Metodologi penelitian dilakukan sesuai dengan yang diilustrasikan pada Gambar **??**. Perancangan sebuah KG dimulai dengan memahami *competency questions* (*requirements*). *Competency questions* mencakup semua pertanyaan yang harus terdapat jawabannya di dalam KG agar dapat digunakan pada *use case* yang akan disebutkan pada subbab selanjutnya. Setelah perntanyaan-pertanyaan sudah jelas, pertanyaan-pertanyaan tersebut dapat dikonversi menjadi SPARQL *query* untuk diperiksa apakah KG mengembalikan jawaban yang sesuai. Kemudian, dirancang ontologi (desain KG) yang dapat menjawab pertanyaan-pertanyaan tersebut. Perancangan ontologi ini mempermudah konstruksi KG.



Gambar 1.3: Metodologi Penelitian

Karena sulit untuk menjamin hasil konversi untuk semua PDF peraturan perundang-undangan, akan terdapat beberapa PDF yang akan dipelihara kualitas konversinya yang selanjutnya akan disebut *maintained PDFs*. Pengembangan sistem konversi dilakukan secara iteratif. Setiap terjadi perubahan pada sistem konversi, akan diperiksa apakah untuk semua *maintained PDFs* memberikan hasil konversi dengan kualitas yang tinggi. Sistem konversi akan menerima input berupa PDF dan mengeluarkan output berupa KG dengan ontologi yang sudah dirancang pada tahap sebelumnya.

Untuk menjamin KG yang dibuat dapat menjawab *competency questions*, dalam penelitian ini dilakukan *use case evaluation* yaitu berupa demonstrasi contoh penerapan dari KG berdasarkan *competency questions*. Setelah semua *competency questions* terpenuhi, dilakukan *large scale evaluation* di mana PDF di luar *maintained PDFs* akan dikonversi kemudian dilakukan sampling pada hasil konversinya untuk dievaluasi secara kualitatif.

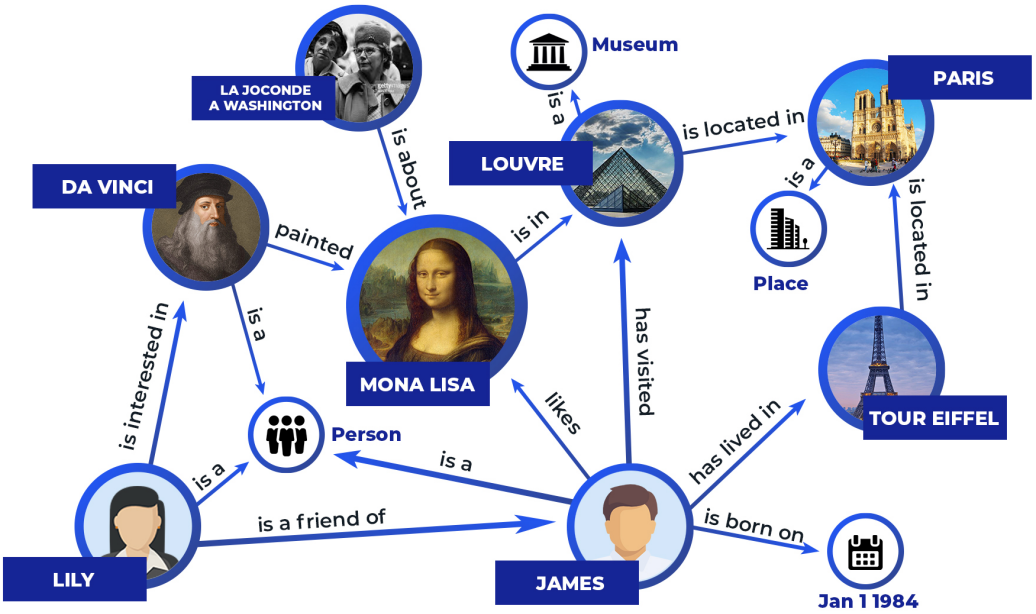
1.5 Sistematika Penulisan

Penelitian ini dibagi ke dalam 6 bab bahasan. Bab pertama menjelaskan mengenai pendahuluan yang berisikan latar belakang serta konteks penelitian yang dilakukan. Pada bab kedua dijelaskan mengenai tinjauan pustaka yang dilakukan di mana dijelaskan berbagai istilah dan bidang ilmu yang berkaitan dengan penelitian yang dilakukan. Lalu, pada bab ketiga dijelaskan mengenai perancangan penelitian yang dilakukan dalam menyusun penelitian ini. Selanjutnya, bab keempat akan membahas implementasi penelitian. Bab kelima akan membahas evaluasi dan analisis hasil penelitian. Bab keenam akan membahas kesimpulan penelitian dan saran untuk penelitan selanjutnya.

BAB 2
TINJAUAN PUSTAKA

2.1 Knowledge Graph

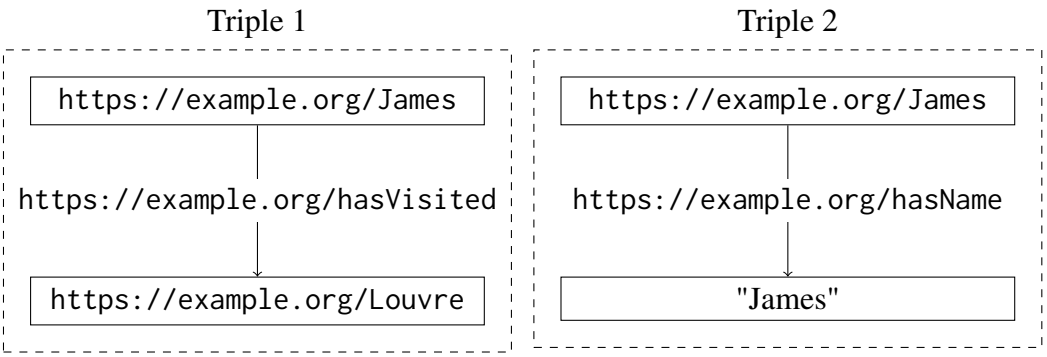
Knowledge graph merupakan model data berbasis *graph* yang menggambarkan entitas dunia nyata dan hubungan antara entitas-entitas tersebut (**?**). Sebagai contoh, pada Gambar **??** dapat dilihat terdapat entitas-entitas yang ditandai dengan bentuk lingkaran, dan hubungan antara entitas-entitas ditandai dengan bentuk anak panah. Pada gambar dapat dilihat bahwa terdapat entitas "James" dan "Louvre" dan properti "has visited" yang mengarah dari "James" ke "Louvre". Relasi tersebut dapat kita anggap sebagai sebuah pengetahuan yang dalam bahasa manusia dapat dituliskan sebagai "James pernah mengunjungi Louvre". Model data seperti ini cocok untuk menyimpan data dengan banyak jenis relasi antara entitas seperti pengetahuan umum, dibandingkan dengan *relational database* yang cocok untuk menyimpan data dengan relasi antar entitas yang sudah diketahui dan terbatas.



Gambar 2.1: Contoh *Knowledge Graph*
Sumber: (**?**)

2.2 Resource Description Framework

Resource Description Framework (RDF) merupkan model data yang memberikan pernyataan tentang suatu *resource* dalam bentuk subjek-properti-objek, atau sering disebut *triple*. Sebuah KG dapat direpresentasikan sebagai kumpulan *triple*. Subyek dan properti dalam *triple* direpresentasikan sebagai Uniform Resource Identifier (URI), sedangkan objek dapat direpresentasikan dengan URI atau *string literal*. Pada Gambar **??**, dapat dilihat terdapat KG dengan dua *triple*. Pada *Triple* 1, subjek, properti, dan objek berturut-turut adalah `https://example.org/James`, `https://example.org/hasVisited`, `https://example.org/Louvre` yang mana semuanya berbentuk URI. Sedangkan pada *Triple* 2, subjek, properti, dan objek berturut-turut adalah `https://example.org/James`, `https://example.org/hasName`, "James". `https://example.org/James`, `https://example.org/Louvre` berturut-turut merupakan notasi untuk *resource* entitas James dan Louvre, `https://example.org/hasVisited` dan `https://example.org/hasName` merupakan properti yang menjelaskan hubungan antara subjek dan objek, dan "James" adalah *string literal* yang menunjukan nilai dari nama James.



Gambar 2.2: Contoh *triple*

Terdapat beberapa format berkas dan sintaks untuk mengekspresikan RDF. Pada penelitian ini, penulis menggunakan format Terse RDF Triple Language (Turtle) (**?**). Sebagai contoh, KG pada Gambar **??** dapat dituliskan dalam format Turtle seperti Kode **??**.

```
<https://example.org/James> <https://example.org/hasVisited>
 ↪  <https://example.org/Louvre> .
<https://example.org/James> <https://example.org/hasName> "James" .
```

Kode 2.1: Contoh penulisan *triple* dalam Turtle

Dengan fitur format Turtle, dua *triple* pada Kode **??** dapat ditulis lebih singkat de-

ngan mengganti prefix URI yang sama menjadi teks yang lebih singkat, dan menggabungkan penulisan beberapa *triple* dengan subjek yang sama sehingga subjek hanya perlu dituliskan sekali saja. Sebagai contoh, karena sebagaian besar URI memiliki prefix `https://example.org/`, kita dapat menggantinya dengan *string* `ex:` dengan mendeklarasikan `@prefix ex: https://example.org/ .`. Selanjutnya `ex:` digunakan sebagai notasi singkat `https://example.org/` untuk mempermudah pembacaan. Kedua *triple* juga memiliki subjek yang sama yaitu `ex:James`, oleh karena itu, penulisan kedua Turtle bisa digabung dengan hanya menuliskan subjek sebanyak satu kali, kemudian menuliskan properti dan objek dipisah dengan tanda titik koma. Kode **??** adalah contoh penulisan Kode **??** dalam bentuk yang lebih singkat dan lebih mudah dibaca oleh manusia.

```
@prefix schema: <http://schema.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<http://data.europa.eu/eli/dir/1980/181/oj> a schema:Legislation ;
  schema:about <http://eurovoc.europa.eu/1896> ;
  schema:isBasedOn <http://njh.me/11957E100.html> ;
  schema:legislationDate "1979-12-20"^^schema:Date ;
  schema:legislationPassedBy "Council of the European Union"^^xsd:string ;
  schema:name "Council Directive 80/181/EEC of 20 December 1979 on the approximation

<http://eurovoc.europa.eu/1896> a schema:Thing ;
  schema:name "Metrology"^^xsd:string .

<http://njh.me/11957E100.html> a schema:Legislation .
```

Kode 2.2: Contoh penulisan lebih singkat sebuah *triple* dalam Turtle

2.3 SPARQL

SPARQL merupakan salah satu RDF *query language* yaitu bahasa yang mampu mengambil atau mengubah data yang disimpan dalam format RDF. Sintaks SPARQL memiliki beberapa kesamaan dengan SQL, contohnya sebuah variabel diawali dengan tanda tanya seperti `?name`. Query SPARQL sederhana dapat dibentuk dengan `SELECT` dan `WHERE` dengan kondisi pola triple ditulis di dalam `WHERE` dan variabel yang akan ditampilkan ditulis didalam `SELECT` (**?**). Berikut adalah penjelasan singkat *clause* SPARQL yang digunakan pada penelitian ini.

- `PREFIX`: Mendeklarasikan pemetaan prefix URI menjadi *string* yang lebih singkat. Hal ini dilakukan agar *query* lebih mudah dibaca oleh manusia seperti sintaks `@prefix` pada Turtle.
- `SELECT`: Menyatakan variabel yang akan ditampilkan sebagai output dari *query*.
- `WHERE`: Menyatakan kondisi *triple* yang akan ditampilkan pada output. Elemen dari *triple* yaitu subjek, predikat, dan objek dapat beruba vaiable atau *fixed value* seperti *string* atau bilangan.
-

Berikut adalah contoh-contoh *query* SPARQL yang dilakukan pada contoh KG pada Kode **??**.

2.3.1 Contoh Query 1

Kode **??** merupakan SPARQL *query* untuk menampilkan semua *triple* yang terdapat pada KG, akan memberikan output Tabel **??**. Setiap baris yang terdapat pada output merupakan *triple* yang tedapat pada KG.

```
PREFIX ex: <https://example.org/>
SELECT ?visited_place
WHERE { ex:James ex:hasVisited ?visited_place }
```

Kode 2.3: Query 1

?subjek	?predikat	?objek
ex:James	ex:hasVisited	ex:Louvre
ex:James	ex:hasName	"James"

Tabel 2.1: Output Query 1

2.3.2 Contoh Query 2

Kode **??** merupakan SPARQL *query* untuk menampilkan semua properti dan objek pada *triple* dengan subjek `ex:James`. pada KG, akan memberikan output Tabel **??**.

```
PREFIX ex: <https://example.org/>
SELECT ?properti ?objek
WHERE { ex:James ?properti ?objek }
```

Kode 2.4: Query 2

?properti	?objek
ex:hasVisited	ex:Louvre
ex:hasName	"James"

Tabel 2.2: Output Query 3

2.3.3 Contoh Query 3

Kode **??** merupakan SPARQL *query* untuk menampilkan semua objek pada *triple* dengan subjek `ex:James` dan properti `ex:hasVisited`, dengan output Tabel **??**. Artinya, *query* ini menampilkan semua tempat yang pernah dikunjungi James, yaitu Louvre.

```
PREFIX ex: <https://example.org/>
SELECT ?visited_place
WHERE { ex:James ex:hasVisited ?visited_place }
```

Kode 2.5: Query 3

?visited_place
ex:Louvre

Tabel 2.3: Output Query 3

2.4 Peraturan perundang-undangan

Menurut Pasal 7 Ayat 1 UU 10/2004 tentang pembentukan peraturan perundang-undangan, peraturan perundang-undangan adalah peraturan tertulis yang dibentuk oleh lembaga negara atau pejabat yang berwenang dan mengikat secara umum. Jenis dan hierarki peraturan perundang-undangan terdiri atas:

1. Undang-Undang Dasar Negara Republik Indonesia Tahun 1945

2. Undang-Undang/Peraturan Pemerintah Pengganti Undang-Undang
3. Peraturan Pemerintah
4. Peraturan Presiden
5. Peraturan Daerah

Berikut adalah beberapa peranan perundang-undangan untuk Indonesia (**?**):

- Peraturan perundang-undangan merupakan kaidah hukum yang mudah dikenal (diidentifikasi), mudah diketemukan kembali, dan mudah ditelusuri. Sebagai kaidah hukum tertulis, bentuk, jenis,dan tempatnya jelas. Begitu pula pembuatnya.
- Peraturan perundang-undangan memberikan kepastian hukum yang lebih nyata karena kaidah-kaidahnya mudah diidentifikasi dan mudah diketemukan kembali.
- Struktur dan sistematika peraturan perundang-undangan lebih jelas sehingga memungkinkan untuk diperiksa kembali dan diuji baik segi-segi formal maupun materi muatannya.
- Pembentukan dan pengembangan peraturan perundang-undangan dapat direncanakan. Faktor ini sangat penting bagi negara-negara yang sedang membangun termasuk membangun sistem hukum baru yang sesuai dengan kebutuhan dan perkembangan masyarakat.

Menurut lampiran UU 10/2004, struktur peraturan perundang-undangan terdiri atas:

- Judul
- Pembukaan
- Batang Tubuh
- Penutup
- Penjelasan (jika diperlukan)
- Lampiran (jika diperlukan)

Bagian judul, pembukaan, dan penutup memuat metadata dari peraturan perundang-undangan tersebut seperti jenis, nomor, tahun pengundangan atau penetapan, nama, jabatan pembentuk, konsiderans, kasar Hukum, peraturan perundang-undangan. Konsiderans memuat uraian singkat mengenai pokok-pokok pikiran yang menjadi latar belakang dan

alasan pembuatan peraturan perundang-undangan. Dasar Hukum memuat dasar kewenangan pembuatan peraturan perundang-undangan dan peraturan perundang-undangan yang memerintahkan pembuatan peraturan perundang-undangan tersebut. Oleh karena itu, kita dapat melihat keterkaitan antara peraturan perundang-undangan dengan melihat konsiderans dan dasar hukum dari peraturan tersebut.

Batang tubuh peraturan perundang-undangan memuat semua substansi peraturan perundang-undangan yang dirumuskan dalam pasal. Pengelompokkan materi peraturan perundang-undangan dapat disusun secara sistematis dalam buku, bab, bagian, dan paragraf atas dasar kesamaan materi. Pasal dapat dirinci ke dalam beberapa ayat. Pasal dan ayat dapat dibuat dalam bentuk kalimat maupun tabulasi rincian. Dalam bentuk tabulasi rincian, setiap rincian harus dapat dibaca sebagai satu rangkaian kesatuan dengan frase pembuka dan setiap rincian diawali dengan huruf (abjad) kecil dan diberi tanda baca titik. Suatu rincian dapat dibagi lagi ke unsur rincian yang lebih kecil.

Amendemen (perubahan) dapat dilakukan pada suatu peraturan perundang-undangan. Secara teoritis, perubahan konstitusi (*constitutional amendment*) mengandung tiga macam arti: 1) Menjadikan lain bunyi kalimatnya; 2) Menambahkan sesuatu yang baru, dan; 3) Ketentuan daman Undang-Undang Dasar dilaksanakan tidak seperti yang tercantum di dalamnya(**?**). Bentuk amendemen dapat berupa pengubahan, penyisipan, dan penghapusan. Amendemen juga dapat dilakukan pada tingkat peraturan maupun komponen peraturan seperti pasal dan bab.

2.5 *European Legislation Identifier* (ELI)

Negara-negara anggota EU memiliki cara mengatur informasi hukum yang berbeda-beda, sehingga cenderung menghalangi penemuan, pertukaran, dan penggunaan kembali informasi. European Legislation Identifier (ELI) adalah solusi yang bertujuan untuk mengatasi masalah-masalah tersebut. ELI merupakan sistem untuk membuat peraturan perundang-undangan tersedia secara daring dalam format terstandardisasi, sehingga dapat diakses dan digunakan oleh berbagai instansi (**?**). ELI bertujuan untuk untuk memfasilitasi akses, berbagi dan interkoneksi informasi hukum yang diterbitkan melalui nasional, Eropa dan sistem informasi hukum global. ELI dibangun berdasarkan persetujuan antara negara-negara EU.

ELI juga memiliki *technical implementation guide*[1] yang menjelaskan implementasi ELI dan spesifikasi yang tedapat pada ELI. Spesifikasi ELI antara lain adalah cara membuat URI suatu *resource*, *class*, dan properti. Ontologi yang digunakan pada ELI cenderung untuk menjelaskan metadata dari peraturan, dan tidak memiliki pemodelan

[1] `https://publications.europa.eu/en/publication-detail/-/publication/8159b75d-5efc-11e8-ab9c-01aa75ed71a1`

khusus untuk menjelaskan *subdivision* dari peraturan. Nyatanya, ELI menggunakan class *eli:LegalResourceSubdivision* untuk mendefinisikan semua *subdivision*, tidak terdapat kelas khusus untuk variasi *subdivision* seperti bab, pasal, dan ayat. Selain itu, ELI dirancang untuk digunakan pada peraturan di Eropa, sehingga tidak memiliki ontologi untuk jenis peraturan di Indonesia seperti "Peraturan Pemerintah".

2.6 Schema.org

Schema.org adalah aktivitas komunitas kolaboratif dengan misi untuk membuat, memelihara, dan mempromosikan skema untuk data terstruktur di internet, di halaman web, dalam pesan email, dan lebih dari itu (**?**). *Vocabulary* Schema.org dapat digunakan dengan *encoding* yang berbeda, termasuk RDFa, Microdata dan JSON-LD. Banyak aplikasi dari Google, Microsoft, Pinterest, Yandex, dan lainnya sudah menggunakan *vocabulary* ini untuk memperkuat pengalaman yang kaya dan dapat diperluas. Schema.org memiliki *vocabulary* tersendiri untuk peraturan perundang-undangan.[2] Salah satu *vocabulary* nya adalah properti yang menjelaskan metadata suatu peraturan perundang-undangan seperti `legislationDate` dan `jurisdiction`.

2.7 Wikidata

Wikidata adalah basis pengetahuan *free* dan terbuka yang dapat dibaca dan diedit oleh manusia dan mesin (**?**). Wikidata berperan sebagai penyimpanan pusat untuk data terstruktur dari proyek saudara Wikimedia termasuk Wikipedia, Wikivoyage, Wiktionary, Wikisource, dan lainnya. WikiProject adalah organisasi yang didirikan untuk mencapai tujuan penyuntingan tertentu, atau untuk mencapai tujuan yang berkaitan dengan bidang pengetahuan tertentu. Terdapat WikiProject tersendiri untuk bidang hukum dan peraturan perundang-undangan yaitu WikiProject Law.[3] WikiProject Law menyediakan properti terkait peraturan perundang-undangan sebagaimana yang dapat dilihat pada Gambar **??**.

[2] `https://schema.org/Legislation`
[3] `https://www.wikidata.org/wiki/Wikidata:WikiProject_Law`

Properties [edit]

V · T · E	Wikidata property related to law and justice (Q22984026)
General	foundational text · main regulatory text · highest judicial authority · legislative body
Laws	legislated by · applies to jurisdiction · amended by · repealed by · repeals · introduced on
Trials	court · judge · prosecutor · defender · plaintiff · defendant · charge · plea · penalty · laws applied · overrules
Treaties	ratified by · depositor · signatory
Identifiers	legal citation of this text · United States Public Law · European Case Law ID · CELEX number · Federal Register Document Number · legislation.gov.uk ID · Old Bailey Proceedings ID · SR Number · Caselaw Access Project case ID · NOR · Ukrainian regulations base ID · United Nations Treaty Series Registration Number · CETS number · Cour des comptes magistrate ID · Shoftim BeIsrael judge ID · Google Scholar case ID

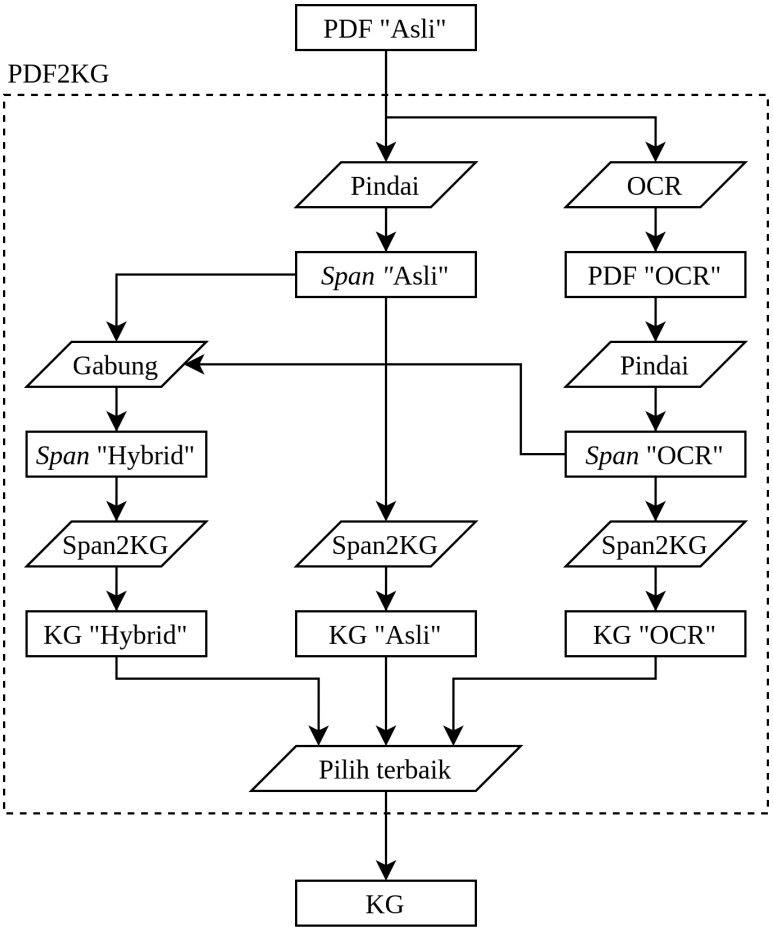
Gambar 2.3: Properti WikiProject Law

BAB 3
PERANCANGAN

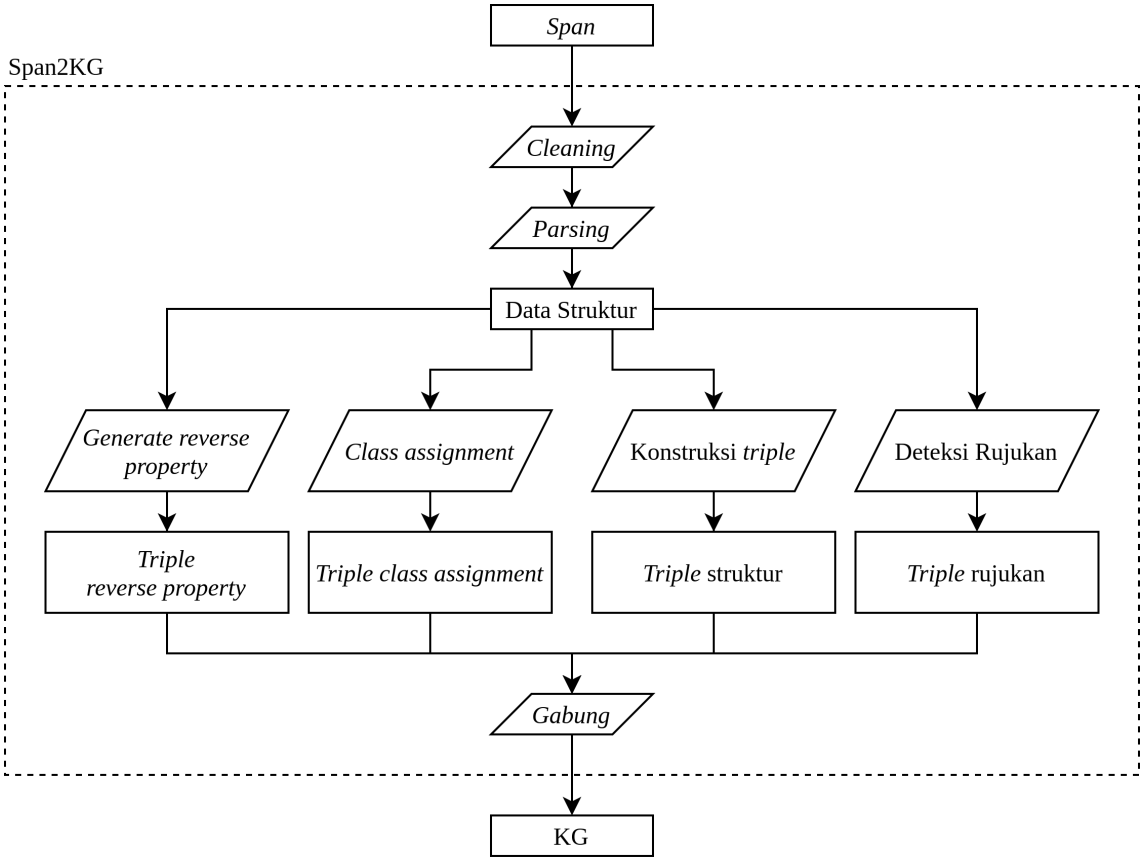
Bab ini berisi perancangan arsitektur konversi, ontologi KG yang dihasilkan, dikaitkan dengan *competency questions*. *Competency questions* harus diperjelas untuk dijadikan landasan ontologi KG. KG yang dihasilkan oleh sistem konversi, digabung dengan ontologi yang sudah dirancang harus dapat menjawab semua pertanyaan pada *competency questions*.

3.1 Arsitektur Sistem Konversi

Jgn lp jg dijelasin step2nya yg ada di arsitektur/diagram tsb Proses konversi dokumen PDF ke KG mengikuti arsitektur seperti yang ditunjukkan pada Gambar **??** dan Gambar **??**. Bagian *Optical Character Recognition* (OCR) bertanggung jawab untuk mengubah *image-only* PDF menjadi *text-scannable* PDF dan menstandardisasi pemformatan semua PDF. Selanjutnya, penulis melakukan pemindaian file PDF untuk mengekstrak teks beserta posisinya. Selanjutnya, setiap baris teks yang dipindai sebagai *span*. Setelah itu, *span* yang terdeteksi sebagai konten yang tidak relevan, seperti *header* dan *footer* yang berulang, akan disaring. *Clean span* yang dihasilkan, kemudian diuraikan menjadi data peraturan perundang-undangan yang bermakna dan terstruktur. Pada tahap akhir, sistem konversi membangun KG peraturan perundang-undangan dari hasil *parsing*. Setiap KG peraturan perundang-undangan dihasilkan dari PDF akan digabung menjadi satu KG global.



Gambar 3.1: Arsitektur PDF2KG



Gambar 3.2: Arsitektur Span2KG

3.2 *Competency Questions*

Competency questions perlu didefinisikan untuk dijadikan landasan perangancan ontologi KG yang dihasilkan. Setiap *competency question* harus dapat dijawab oleh KG yang dihasilkan dengan membuktikan SPARQL *query* yang bersesuaian dengan pertanyaan tersebut mengembalikan hasil yang benar. Tabel **??** merupakan daftar *competency questions* yang didefinisikan pada penelitian ini.

No.	**Pertanyaan**
1	Deskripsikan UU Cipta Kerja.
2	Tampilkan semua pasal dari UU Cipta Kerja.
3	Tampilkan teks dari Pasal 2 Ayat 1 UU Cipta Kerja.
4	Tampilkan semua pasal yang terdapat pada Bab 2 UU Cipta Kerja.
5	Tampilkan ayat yang mengandung teks "kompensasi" dan "buruh" yang disisipkan oleh UU Cipta Kerja.
6	Tampilkan komponen dari UU Cipta Kerja yang menyisipkan pasal ke UU Ketenagakerjaan dan pula teks dari pasal tersebut.

No.	Pertanyaan
7	Tampilkan komponen dari UU Cipta Kerja yang mengubah pasal UU Ketenagakerjaan dan tambilkan sebelum dan setelah pengubahan.
8	Tampilkan komponen dari UU Cipta Kerja yang menghapus pasal UU Ketenagakerjaan dan pula teks dari pasal tersebut.
9	Tampilkan peraturan yang paling banyak diamendemen oleh UU Cipta Kerja.
10	Tampilkan banyaknya penyisipan, pengubahan, dan penghapusan pasal yang dilakukan UU Cipta Kerja terhadap UU lain.
11	Tampilkan pasal pada UU Ketenagakerjaan setelah dilakukan oleh UU Cipta Kerja.
12	Tampilkan pasal pada UU Cipta Kerja yang tidak melakukan amendemen.
13	Tampilkan pasal pada UU Cipta Kerja yang menghapus pasal yang disahkan setelah tahun 2001.
14	Tampilkan pasal yang disisipkan oleh UU Cipta Kerja dan rujukan yang terdapat pada pasal jika ada.
15	Tampilkan semua peraturan serta peraturan yang ditimbangnya.
16	Tampilkan 10 peraturan dengan pasal terbanyak.
17	Tampilkan peraturan yang pernah diamendemen.
18	Tampilkan 10 tempat terbanyak peraturan disahkan.
19	Tampilkan 10 peraturan paling banyak ditimbang.
20	Tampilkan semua peraturan yang melakukan amendemen dan banyaknya pasal yang amendemen oleh pasal tersebut.
21	Tampilkan jumlah *triple* tentang struktur peraturan.
22	Tampilkan jumlah *triple* tentang metadata peraturan.
23	Tampilkan jumlah *triple* tentang teks peraturan.
24	Tampilkan jumlah *triple* tentang rujukan peraturan.
25	Tampilkan jumlah *triple* tentang amendemen peraturan.
26	Tampilkan jumlah *triple* tentang *class assignment*.
27	Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya.
28	Peraturan apa yang berlaku pada daerah X yang disahkan di tahun Y?
29	Apa relasi suatu peraturan dengan peraturan lain?
30	Mana saja peraturan yang membahas tentang topik W atau X tapi bukan topik Y dan bukan Z?
31	Apa isi dari Pasal X ayat (Y) huruf Z dari suatu peraturan perundang-undangan?
32	Apa saja bab-bab yang terdapat pada peraturan X?

Tabel 3.1: *Competency questions*

3.3 *Lex2KG namespace*

Setiap entitas yang berbeda perlu diberikan URI yang berbeda. Penggabungan KG sangat umum dilakukan karena secara struktur dapat dilakukan dengan mudah, dan dapat memperkuat data dari masing-masing KG. KG memiliki entitas-entitas dengan URI yang unik, oleh karena itu saat menggabungkan sebuah KG A dan sebuah KG B, entitas yang berbeda harus memiliki URI yang berbeda. Sebuah masalah dapat terjadi apabila URI tidak dirancang dengan benar. Sebagai contoh, apabila terdapat entitas dari KG A dengan URI `https://example.org/trump` dengan maksud Donald Trump mantan presiden Amerika Serikat, dan entitas dari KG B dengan URI yang sama dengan maksud permainan kartu Trump, saat kedua KG ini digabung akan menjadi informasi yang berkontradiksi. Oleh karena itu, pada umumnya digunakan *namespace* yang berbeda untuk masing-masing KG. Sebagai contoh, dengan KG A menggunakan *namespace* `https://kga.org` dan KG B menggunakan *namespace* `https://kgb.org`, dua entitas yang sebelumnya berkontradiksi akan memiliki URI yang berbeda yaitu `https://kga.org/trump` dan `https://kgb.org/trump`.

Pada penelitian ini digunakan *namespace* `https://example.org/lex2kg/`. *Namespace* yang digunakan oleh Lex2KG dapat dikonfigurasi. Tabel **??** adalah daftar *prefix* dan notasi singkatnya pada yang akan digunakan pada selanjutnya.

notasi singkat	*prefix*
`:`	`https://example.org/lex2kg/`
`o:`	`https://example.org/lex2kg/ontology/`
`jp:`	`https://example.org/lex2kg/ontology/jenisPeraturan`
`uu:`	`https://example.org/lex2kg/uu/`

Tabel 3.2: *Prefix* dan notasi singkatnya

3.4 Perancangan *Class* dan URI *Class*

Huruf pertama dari nama *class* dibuat kapital mengikuti konvensi RDF (**?**). *Class* Lex2KG terdiri dari *Class* komponen peraturan perundang-undangan seperti `o:Bab` dan *Class* peraturan perundang-undangan sendiri yaitu `o:Peraturan`. Tabel **??** adalah semua URI *class* Lex2KG beserta URI dan deskripsinya.

URI Class	**Deskripsi**
`o:Peraturan`	Peraturan perundang-undangan
`o:DaftarBab`	Daftar satu atau lebih bab
`o:Bab`	Bab

URI Class	Deskripsi
`o:DaftarBagian`	Daftar satu atau lebih bagian
`o:Bagian`	Bagian
`o:DaftarParagraf`	Daftar satu atau lebih paragraf
`o:Paragraf`	Paragraf
`o:DaftarPasal`	Daftar satu atau lebih pasal
`o:Pasal`	Pasal
`o:VersiPasal`	Versi dari sebuah pasal. Sebuah pasal dapat memiliki satu atau lebih versi dari hasil amendemen
`o:DaftarAyat`	Daftar satu atau lebih ayat
`o:Ayat`	Ayat
`o:DaftarHuruf`	Daftar satu atau lebih huruf atau nomor
`o:Huruf`	Huruf atau nomor
`o:Menimbang`	Hal yang ditimbang oleh suatu peraturan
`o:Mengingat`	Hal yang diingat oleh suatu peraturan
`o:Segmen`	Segmen teks. Dapat memiliki rujukan ke suatu komponen peraturan

Tabel 3.3: Class Lex2KG

3.5 Perancangan Jenis Peraturan dan URI Jenis Peraturan

Setiap peraturan memiliki URI yang berbeda, yang akan digunakan oleh entitas sebagai prefix. Tabel **??** adalah jenis peraturan perundang-undangan yang diimplementasi pada penelitian ini beserta URI nya. Variabel akan ditandai dengan kurung kurawal "`{}`". Diluar jenis peraturan yang diimplementasi URI nya, ontologi Lex2KG juga memiliki resource sendiri yang mendefinisikan jenis dari peraturan perundang-undangan terdaftar pada Tabel **??**.

Peraturan	URI	Penjelasan variabel
Undang-Undang Dasar 1945	`:uud`	
Undang-Undang	`:uu/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Pemerintah	`:pp/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan

Peraturan	URI	Penjelasan variabel
Peraturan Daerah Provinsi DKI Jakarta	`:perda_provinsi_dki_jakarta/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Gubernur DKI Jakarta	`:pergub_dki_jakarta/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan
Peraturan Walikota Malang	`:perwali_malang/{thn}/{nmr}`	`thn`: tahun peraturan `nmr`: nomor peraturan

Tabel 3.4: Konstruksi URI untuk peraturan perundang-undangan

Jenis Peraturan	URI
Undang-Undang Dasar 1945	`jp:UUD`
Ketetapan MPR	`jp:TapMPR`
Undang-Undang	`jp:UU`
Peraturan Pemerintah Pengganti Undang-Undang	`jp:Perpu`
Peraturan Pemerintah	`jp:PP`
Peraturan Presiden	`jp:Perpres`
Peraturan Menteri	`jp:PeraturanMenteri`
Peraturan Daerah Provinsi	`jp:PerdaProvinsi`
Peraturan Gubernur	`jp:PeraturanGubernur`
Peraturan Daerah Kabupaten/Kota	`jp:PerdaKabKota`
Peraturan Bupati/Walikota	`jp:PeraturanBupatiWalikota`
Keputusan Presiden	`jp:KeputusanPresiden`
Keputusan Gubernur	`jp:KeputusanGubernur`
Keputusan Menteri	`jp:KeputusanMenteri`
Keputusan Bupati/Walikota	`jp:KeputusanBupatiWalikota`

Tabel 3.5: URI jenis peraturan perundang-undangan

3.6 Perancangan Komponen dan URI Komponen

Setiap komponen dokumen merupakan entitas, dan memiliki URI. URI sebuah komponen didahului oleh *docURI* dan semua komponen yang mengatasinya. Tabel **??** adalah jenis komponen peraturan perundang-undangan beserta pola URI dan penjelasan variabelnya.

Komponen	URI	Penjelasan variabel
o:DaftarBab	{docURI}/bab	docURI: URI peraturan yang mengatasi
o:Bab	{daftarBabURI}/{no}	daftarBabURI: URI o:DaftarBab yang mengatasi no: Nomor bab
o:DaftarBagian	{babURI}/bagian	babURI: URI o:Bab yang mengatasi
o:Bagian	{daftarBagianURI}/{no}	daftarBagianURI: URI o:DaftarBagian yang mengatasi no: Nomor bagian
o:DaftarParagraf	{bagianURI}/paragraf	bagianURI: URI o:Bagian yang mengatasi
o:Paragraf	{daftarParagrafURI}/{no}	daftarParagrafURI: URI o:DaftarBagian yang mengatasi no: Nomor paragraf
o:DaftarPasal	{parentURI}/daftarpasal	parentURI: URI o:Bab, o:Bagian, o:Paragraf yang mengatasi
o:Pasal	{docURI}/pasal/{no}	docURI: URI peraturan yang mengatasi no: Nomor pasal
o:VersiPasal	{pasalURI}/versi/{tanggal}	pasalURI: URI o:Pasal yang mengatasi tanggal: Tanggal disahkan
o:DaftarAyat	{pasalURI}/ayat	pasalURI: URI o:Pasal yang mengatasi
o:Ayat	{daftarAyatURI}/{no}	daftarAyatURI: URI o:DaftarAyat yang mengatasi no: Nomor ayat
o:DaftarHuruf	{parentURI}/huruf	parentURI: URI o:Ayat, o:Huruf, o:VersiPasal, o:Menimbang, o:Mengingat, yang mengatasi
o:Huruf	{daftarHurufURI}/{id}	daftarHurufURI: URI o:DaftarHuruf yang mengatasi id: Nomor atau huruf
o:Menimbang	{docURI}/menimbang	docURI: URI peraturan yang mengatasi
o:Mengingat	{docURI}/mengingat	docURI: URI peraturan yang mengatasi

Komponen	URI	Penjelasan variabel
o:Segmen	{parentURI}/{name}	parentURI: URI o:Ayat, o:Huruf, o:DaftarHuruf, o:VersiPasal, o:Menimbang, Mengingat, yang mengatasi name: Nama teks

Tabel 3.6: Konstruksi URI untuk komponen peraturan perundang-undangan

3.7 Perancangan Properti dan URI Properti

Properti menghubungkan subjek dan objek. Subjek adalah berupa URI komponen, sedangkan objek dapat berupa URI komponen, *string*, , atau tanggal. Tipe data atau *class* yang memungkinkan untuk subjek dari sebuah properti disebut *domain*, sedangkan hal yang sama untuk objek disebut *range*. Tabel **??** adalah properti Lex2KG berupa nama, *domain*, *range*, dan deskripsinya.

Nama	Domain	Range	Deskripsi
o:nomor	o:Bab, o:Bagian, o:Paragraf, o:Pasal, o:Ayat, o:Huruf	string, bilangan	Nomor atau huruf _identifier_ dari komponen
o:teks	o:Ayat, o:VersiPasal, o:Segmen	string	teks dari komponen
o:bab	o:DaftarBab	o:Bab	Bab
o:bagian	o:DaftarBagian	o:Bagian	Bagian
o:paragraf	o:DaftarParagraf	o:Paragraf	Paragraf
o:pasal	o:DaftarPasal	o:Pasal	Pasal
o:ayat	o:DaftarAyat	o:Ayat	Ayat
o:huruf	o:DaftarHuruf	o:Huruf	Huruf
o:segmen	o:Mengingat, o:Menimbang, o:Paragraf, o:VersiPasal, o:Huruf, o:DaftarHuruf	o:Segmen	Segmen
o:daftarBab	o:Peraturan	o:DaftarBab	DaftarBab

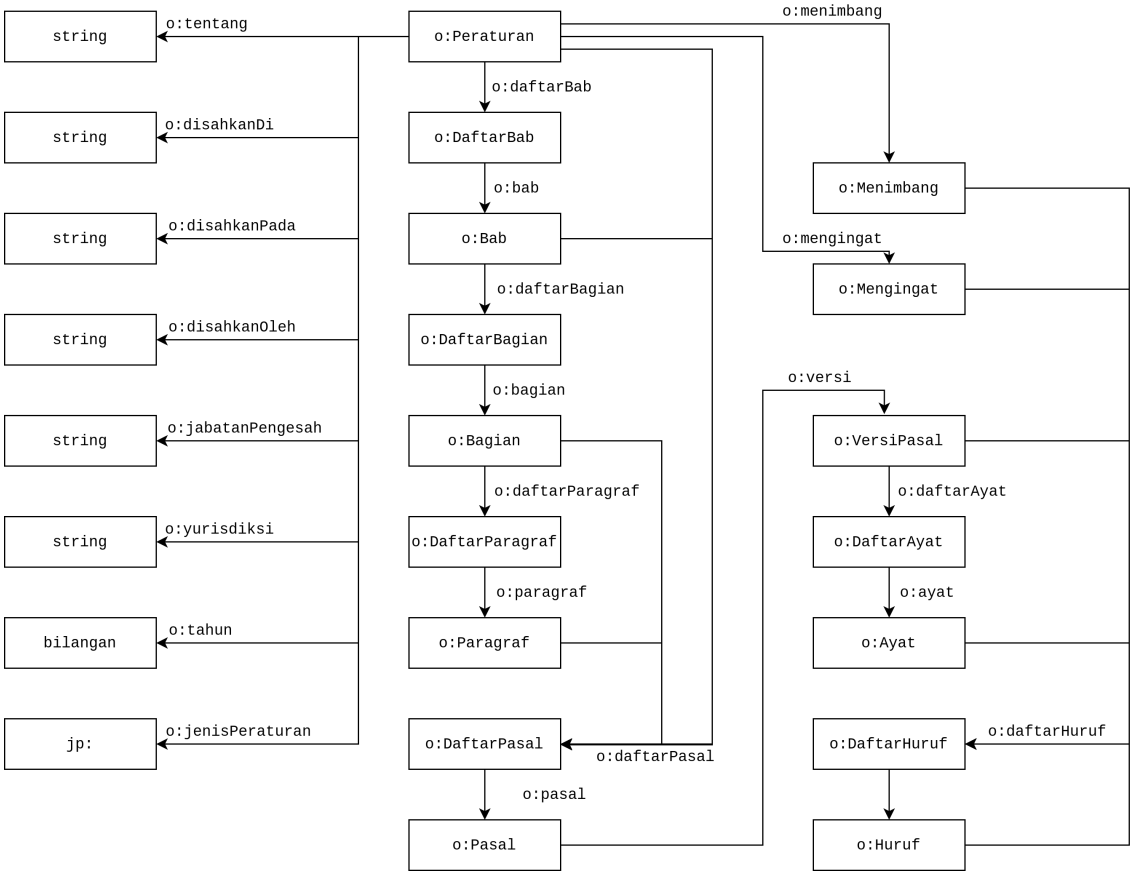
Nama	Domain	Range	Deskripsi
o:daftarBagian	o:Bab	o:Daftarbagian	DaftarBagian
o:daftarParagraf	o:Bagian	o:DaftarParagraf	DaftarParagraf
o:daftarPasal	o:Peraturan,o:Bab, o:Paragraf, o:Bagian	o:DaftarPasal	DaftarPasal
o:daftarAyat	o:VersiPasal	o:DaftarAyat	DaftarAyat
o:daftarHuruf	o:VersiPasal, o:Ayat, o:Huruf, o:Menimbang, o:Mengingat	o:DaftarHuruf	DaftarHuruf
o:merujuk	o:Segmen	entitas apapun	Teks merujuk suatu entitas
o:mengubah	o:Huruf	o:VersiPasal	Pengubahan pasal
o:menghapus	o:Huruf	o:VersiPasal	Penghapusan pasal
o:menyisipkan	o:Huruf	o:VersiPasal	Penyisipan pasal
o:tanggal	o:VersiPasal	tanggal	Tanggal
o:jenisVersi	o:VersiPasal	'orisinal', 'penyisipan', 'pengubahan', 'penghapusan'	Jenis versi
o:versi	o:Pasal	o:VersiPasal	Versi suatu komponen
o:tentang	o:Peraturan	string	Peraturan tentang
o:menimbang	o:Peraturan	o:Menimbang	Peraturan menimbang
o:mengingat	o:Peraturan	o:Mengingat	Peraturan mengingat
o:disahkanPada	o:Peraturan	tanggal	Peraturan disahkan pada tanggal
o:disahkanDi	o:Peraturan	string	Peraturan disahkan di lokasi
o:disahkanOleh	o:Peraturan	string	Peraturan disahkan oleh
o:jabatanPengesah	o:Peraturan	string	Jabatan pengesah peraturan
o:bagianDari	komponen apapun	komponen apapun	Komponen subjek adalah bagian dari komponen objek
o:yurisdiksi	o:Peraturan	string	Komponen subjek adalah bagian dari komponen objek
o:jenisPeraturan	o:Peraturan	Jenis Peraturan (jp:)	Komponen subjek adalah bagian dari komponen objek
o:tahun	o:Peraturan	bilangan	Komponen subjek adalah bagian dari komponen objek

Nama	Domain	Range	Deskripsi
`o:judul`	`o:Bab`, `o:Bagian`, `o:Paragraf`	string	Komponen subjek adalah bagian dari komponen objek

Tabel 3.7: Properti Lex2KG

3.8 Ilustrasi Ontologi

Untuk memahami ontologi secara intuitif, ontologi dapat divisualisasikan. Gambar **??** adalah ontologi struktur data peraturan yaitu komponen-komponen peraturan serta properti yang menghubungkannya dan metadata yang terdapat pada peraturan.



Gambar 3.3: Ontology dari metadata dan struktur peraturan

BAB 4
IMPLEMENTASI

Bab ini berisi penjelasan spesifik proses implementasi pada setiap tahap. Secara garis besar implementasi Lex2KG terdiri dari OCR, pemindaian menjadi *span*, membersihkan *span*, *parsing span* menjadi komponen, deteksi rujukan, dan konstruksi. Selain itu dibahas juga mengenai bagaimana kualitas konversi dipertahankan dan bagaimana melakukan SPARQL *query* untuk melakukan evaluasi. Seluruh algoritma dan program yang digunakan dijelaskan secara rinci dalam bab ini. Seluruh program dan *resource* tersedia pada repositori Lex2KG.[1]

4.1 *Maintained PDFs*

Maintained PDFs merupakan berkas-berkas PDF peraturan perundang-undangan yang akan dipelihara kualitas konversinya. Pengembangan sistem konversi dilakukan secara iteratif, dan setiap terjadi perubahan pada sistem konversi, akan diperiksa apakah untuk semua *maintained PDFs* memberikan hasil konversi dengan kualitas yang tinggi. Pemeriksaan hasil konversi dilakukan secara manual dengan melihat satu per satu *diff* dari hasil konversi. Berikut adalah daftar peraturan *maintained PDFs*.

- Peraturan Daerah Provinsi Jakarta Nomor 2 Tahun 2020
- Peraturan Gubernur DKI Jakarta Nomor 33 Tahun 2020
- Peraturan Gubernur DKI Jakarta Nomor 3 Tahun 2021
- Peraturan Walikota Malang Nomor 19 Tahun 2020
- Peraturan Pemerintah Nomor 74 Tahun 2020
- Peraturan Pemerintah Nomor 34 Tahun 2021
- Undang-Undang Nomor 12 Tahun 1997
- Undang-Undang Nomor 13 Tahun 2003
- Undang-Undang Nomor 6 Tahun 2018
- Undang-Undang Nomor 1 Tahun 2020

[1] `https://github.com/aabccd021/legal-kg`

- Undang-Undang Nomor 11 Tahun 2020

4.2 OCR Ulang Berkas PDF

Dokumen peraturan perundang-undangan yang digunakan pada penelitian didapatkan dalam format berkas PDF. Penulis pada awalnya mencoba langsung mengekstraksi data dari PDF, tetapi penulis menemukan kesulitan. Salah satu kesulitan yang penulis temui adalah terdapatnya salah pemindaian pada berkas PDF. Sebagai contoh, terdapat teks yang tertulis `Pasal` tetapi mengandung data `Pasai`. Selain itu, penulis juga menemukan bahwa setiap dokumen mengandung keslahan pemindaian yang bervariasi. Untuk suatu kasus, teks `(2)` selalu terdeteksi sebagai `(21` pada suatu dokumen, tetapi tidak pada dokumen lainnya. Pada kasus lainnya terdapat dokumen yang hampir tidak memiliki kesalahan pemindaian dan juga terdapat dokumen yang teksnya tidak dapat dibaca samasekali.

Walaupun dokumen-dokumen tersebut dipelihara oleh satu lembaga pada satu situs web, yaitu oleh Dewan Perwakilan Rakyat pada situs web `https://dpr.go.id/jdih`, masing-masing dokumen dibuat menjadi berkas PDF dengan cara yang berbeda-beda. Penulis tidak dapat mengetahui secara pasti metode apa yang digunakan, tetapi dari metadata yang didapatkan dari berkas PDF, penulis dapat membuat beberapa dugaan. Pada berkas PDF, terdapat metadata dengan nama **Creator**, di mana pada dokumen-dokumen peraturan perudang-undangan yang didapatkan, tercantum nama-nama alat pencetak atau merk dari pencetak tersebut. Dari informasi tersebut penulis menduga bahwa terdapat sebagian peraturan perundang-undangan yang dibuat menjadi PDF dengan mencetaknya menjadi kertas terlebih dahulu kemudian dipindai oleh pemindai dan sebagian lainnya dikonversi langsung dari berkas *.docx*. Data yang dipindai adalah berupa gambar, artinya teks yang terdapat pada berkas PDF adalah hasil OCR (*optical character recognition*) oleh pemindai tersebut. Tabel **??** adalah salah satu contoh dokumen beserta data yang tercantum sebagai **Creator** dan contoh kesalahan pemindaiannya yang banyak terjadi.

Dokumen	__Creator__	Kesalahan pemindaian
UU 13/2003	ScanSoft PDF Create! 4	- (tidak ada)
UU 6/2018	Canon	‘(2)‘ selalu dipindai ‘(21‘
PP 34/2021	Fuji Xerox B9100	data teks tidak terbaca

Tabel 4.1: Metadata PDF peraturan perundang-undangan

Pemindaian berkas PDF dengan metode yang berbeda-beda memberikan kualitas hasil pemindaian yang berbeda-beda dan kesalahan pemindaian yang tidak konsisten. Untuk menyelesaikan masalah ini, penulis melakukan OCR ulang terhadap semua dokumen

yang akan dikonversi. Dengan melakukan OCR ulang menggunakan satu metode yang sama, penulis tidak hanya berhasil mendapatkan data hasil pemindaian berkas PDF dengan kualitas pemindaian yang konsisten untuk semua dokumen. Penulis memilih menggunakan Tesseract OCR[2] sebagai metode OCR karena sifatnya *open source* dan mendukung Bahasa Indonesia sebagai bahasa yang dipindai.

Kode **??** merupakan *command* untuk melakukan konversi `UU-2021-34.pdf` yang sudah ada menjadi `UU-2021-34_OCR.pdf` hasil OCR. Opsi `-force-ocr` diperlukan untuk melakukan OCR ulang pada berkas PDF yang sudah memiliki teks hasil OCR untuk di *overwrite*. Opsi `-jobs 4` menjalankan *command* dengan 4 *core* CPU, bilangan pada opsi ini dapat diganti sesuai kebutuhan. Opsi `-tesseract-config tesseract-config.cfg` menjalankan *command* dengan konfigurasi yang diberikan.

```
ocrmypdf -l ind --force-ocr --jobs 4 --tesseract-config
↪  tesseract-config.cfg UU-2021-34.pdf UU-2021-34_OCR.pdf
```

Kode 4.1: Command untuk menjalankan OCR

Konfigurasi Tesseract OCR yang digunakan adalah `tessedit_pageseg_mode 4` seperti yang terlihat pada isi file konfigurasi pada Kode **??**. `tessedit_pageseg_mode 4` digunakan agar Tesseract OCR mendeteksi teks sebagai *block*, bukan *word* atau *character*.

```
# tesseract-config.cfg
tessedit_pageseg_mode 4
```

Kode 4.2: Konfigurasi Tesseract OCR

4.3 Pemindaian Berkas PDF menjadi Data *Span*

Agar dapat diproses oleh program, berkas PDF perlu diolah menjadi data berupa daftar teks dan posisinya yang selanjutnya akan disebut *span*. Sebuah *span* mengandung satu baris teks. Tabel **??** adalah data yang dimiliki oleh sebuah *span*. Gambar **??** dan Kode **??** berturut-turut adalah contoh gambaran berkas PDF dan hasil pemindaiannya menjadi daftar *span*.

[2] `https://github.com/tesseract-ocr/tesseract`

Nama	Deskripsi
`str`	teks yand dikandung
`xL`	koordinat titik terkiri dari *span*
`xR`	koordinat titik terkanan dari *span*
`y`	koordinat titik teratas dari *span*
`pageNum`	nomor halaman
`id`	*identifier* unik untuk setiap span

Tabel 4.2: Data *span*

UNDANG-UNDANG REPUBLIK INDONESIA
NOMOR 13 TAHUN 2003
TENTANG
KETENAGAKERJAAN

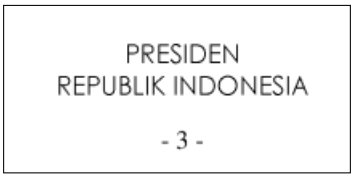
Gambar 4.1: Contoh PDF yang akan dipindai

```yaml
- xL: 197.52
  xR: 442.79990000000004
  'y': 234.48000000000002
  str: UNDANG-UNDANG REPUBLIK INDONESIA
  pageNum: 1
  id: 0
- xL: 252.72
  xR: 387.6002
  'y': 255.12
  str: NOMOR 13 TAHUN 2003
  pageNum: 1
  id: 1
- xL: 291.12
  xR: 349.68048
  'y': 275.76
  str: TENTANG
  pageNum: 1
  id: 2
- xL: 257.28
  xR: 383.28015999999997
  'y': 296.4
  str: KETENAGAKERJAAN
  pageNum: 1
  id: 3
```


Kode 4.3: Hasil pemindaian Gambar **??** (format yaml)

4.4 Membersihkan *Noise* dari Halaman

Tidak jarang dokumen peraturan perundang-undangan mengandung data *noise* yang tidak ingin kita ekstraksi seperti *header* dan *footer*. Pada penelitian ini, penulis menggunakan dokumen dari sumber yang sama sehingga memiliki format yang sama, dan juga posisi *header* dan *footer* yang hampir sama pada setiap dokumen.

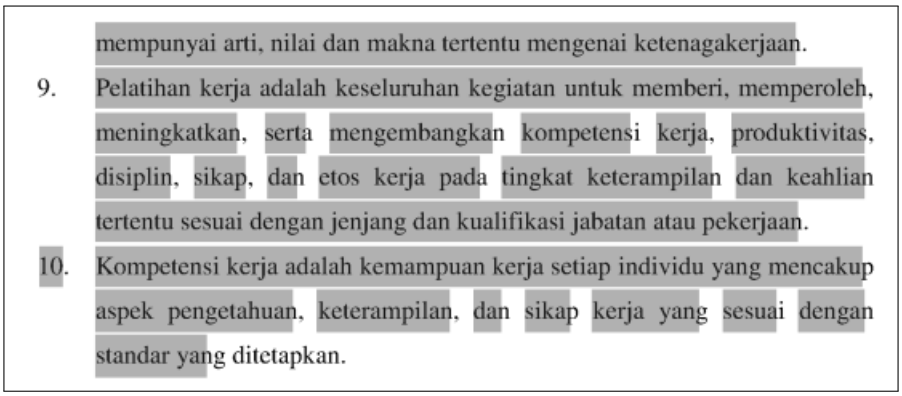


Gambar 4.2: Contoh header

Gambar **??** adalah contoh *header* yang terdapat pada setiap halaman. Dapat dilihat bahwa *header* selalu terdiri dari teks "PRESIDEN REPUBLIK INDONESIA" dan diikuti oleh nomor halaman, dan selalu terletak di posisi yang hampir sama. Oleh karena itu, penulis memeriksa teks menggunakan *regex* dan posisi dari setiap *span*, kemudian menghapus *span* tersebut jika terdeteksi sebagai header.

4.5 Penggabungan Data Berkas PDF Asli dan Hasil OCR Ulang

Pada proses pemindaian berkas PDF menjadi daftar *span*, penulis menemukan satu masalah yaitu data hasil OCR tidak konsisten dalam memindai angka. Seperti yang dapat dilihat pada Gambar **??**, angka 10 berhasil dipindai tetapi angka 9 tidak berhasil. Untuk kasus tersebut, angka hanya tidak terpindai pada hasil OCR ulang, tetapi terpindai dengan benar pada berkas PDF aslinya. Untuk menyelesaikan masalah tersebut, penulis melakukan pemindaian data pada berkas PDF aslinya, kemudian menggabungkannya dengan data hasil pemindaian PDF hasil OCR untuk melengkapi bagian yang tidak terpindai.


mempunyai arti, nilai dan makna tertentu mengenai ketenagakerjaan.

9. Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh, meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin, sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai dengan jenjang dan kualifikasi jabatan atau pekerjaan.

10. Kompetensi kerja adalah kemampuan kerja setiap individu yang mencakup aspek pengetahuan, keterampilan, dan sikap kerja yang sesuai dengan standar yang ditetapkan.

Gambar 4.3: Contoh nomor tidak terpindai

Penulis hanya menggabungkan data dua berkas PDF untuk kasus angka seperti yang disebutkan di atas. Hal tersebut dikarenakan pada umumnya kita dapat mentolerir kesalahan pemindaian pada teks, tetapi karena kegagalan pemindaian pada angka tersebut akan mempengaruhi struktur dokumen, penulis memutuskan solusi di atas. Sebagai contoh, jika nomor 8 berhasil dipindai dan nomor 9 tidak berhasil, maka struktur akhir yang dihasilkan akan mengandung semua isi nomor 9 didalam nomor 8, yang mana seharusnya

adalah nomor yang terpisah. Kode **??** dan Kode **??** berturut-turut adalah contoh data yang dihasilkan jika nomor 9 tidak berhasil terpindai dan jika berhasil terpindai.

```yaml
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
    9. Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

Kode 4.4: Data jika nomor 9 tidak berhasil terpindai (format yaml)

```yaml
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
- type: point
  key: 9
  text: >-
    Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

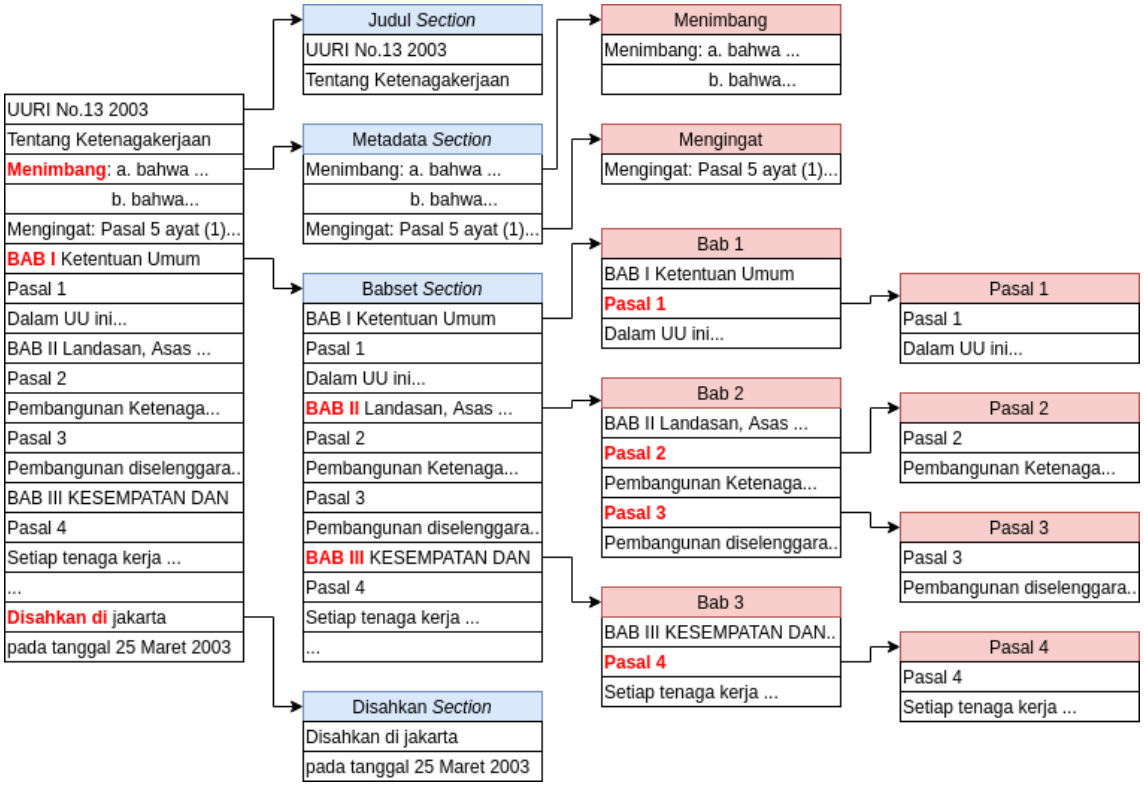
Kode 4.5: Data jika nomor 9 terpindai (format yaml)

4.6 Pengelompokan *span* menjadi Komponen

Teks yang membentuk suatu komponen terdiri dari satu atau lebih *span*, sehingga daftar *span* yang didapatkan dari hasil ekstraksi berkas PDF harus dikelompokkan sehingga setiap kelompok merepresentasikan *span* yang terdapat pada suatu komponen. Pengelompokan dilakukan dengan mendeteksi *span* awal dan *span* akhir dari sebuah komponen. Karena daftar *span* hasil ekstraksi bersifat 1 dimensi, hal ini bisa dilakukan dengan melakukan iterasi pada setiap *span*, kemudian menandai awal atau akhir dari sebuah kelompok

apabila memenuhi suatu pola. Setelah ditandai, *span* akan dikelompokkan sebagai daftar *span* dari suatu komponen.

Pengelompokan tidak langsung dilakukan dari daftar *list* menjadi daftar komponen, melainkan dibuat pengelompokan *section* untuk pembagian dokumen secara garis besar, kemudian baru dikelompokkan sebagai komponen dari masing-masing *section* tersebut. Pada Gambar **??** *section* ditandai dengan warna biru dan komponen ditandai dengan warna merah. Dapat dilihat bahwa daftar *span* data awal dikelompokkan menjadi beberapa *section* yaitu `judul`, `metadata`, `babset`, dan `disahkan`. Kemudian *span* pada section `babset` dikelompokkan menjadi komponen bab yaitu `Bab 1`, `Bab 2`, dan `Bab 3`. Kemudian *span* pada setiap komponen bab dikelompokkan menjadi komponen pasal. Pengelompokan bertingkat ini dilakukan agar fungsi untuk mendeteksi batas antara komponen tidak menjadi rumit.



Gambar 4.4: Ilustrasi Ekstraksi Daftar Span menjadi Data Struktur Komponen

Deteksi batas antara komponen atau *section* dilakukan dengan melihat data pada *span* seperti `str`, `xL`, `xR`, dan `y`. Pada Gambar **??**, pola yang terdeteksi sebagai batas ditandai dengan teks merah. Sebagai contoh, pada ekstraki paling kiri, dari daftar *span* asli menjadi *section*, dilakukan iterasi *span* dari atas dan akan dimasukkan ke dalam *section* `judul` sampai menemukan *section* yang diawali dengan kata **Menimbang**. Kemudian setiap *span* setelah span **Menimbang** tersebut akan dikelompokkan menjadi *section* `metadata` sampai menemukan section yang diawali kata **BAB I**.

4.6.1 Komponen Terurut

Beberapa komponen seperti `bab`, `bagian`, `paragraf`, `pasal`, `angka`, dan `huruf` memiliki sifat terurut. Dalam hal ini, yang termasuk terurut adalah:

- Komponen yang mengawali suatu komponen terurut akan selalu sama.
- Hanya terdapat satu komponen yang dapat mengikuti komponen terurut.

Contoh dari sifat pertama adalah `bab`, `bagian`, `paragraf`, `pasal`, `angka` pasti dimulai dari komponen nomor 1 seperti `bab 1` dan `pasal 1`, dan `huruf` pasti dimulai dari komponen `huruf a`. Contoh dari sifat kedua adalah hanya `pasal 2` yang dapat mengikuti `pasal 1` dan hanya `huruf b` yang dapat mengikuti `huruf a`. Dalam implementasi, penulis memastikan sifat-sifat ini dipatuhi, sehingga sebagai contoh apabila sebuah dokumen terdeteksi mengandung `bab 12` maka akan dijamin mengandung semua `bab 1` sampai `bab 11` dan apabila terdeteksi mengandung `pasal 196` maka akan dijamin mengandung semua pasal dari `pasal 1` sampai `pasal 195`.

4.6.2 Komponen Amendemen

Mengekstraksi *span* pada bagian amendemen merupakan salah satu tantangan pada penelitian ini. Dibawah adalah gambar amendemen pada Pasal 17 UU 11/2020 yang dilakukan terhadap Pasal 1 UU 26/2007. Dapat dilihat bahwa kata `Pasal 17` dan `Pasal 1` yang digunakan sebagai pembatas antara komponen memiliki pola teks dan posisi yang hampir sama. Hal ini membuat pembedaan antara “Pasal biasa” dan “Pasal peng-amendemen” menjadi sulit dilakukan.

Pasal 17

Beberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

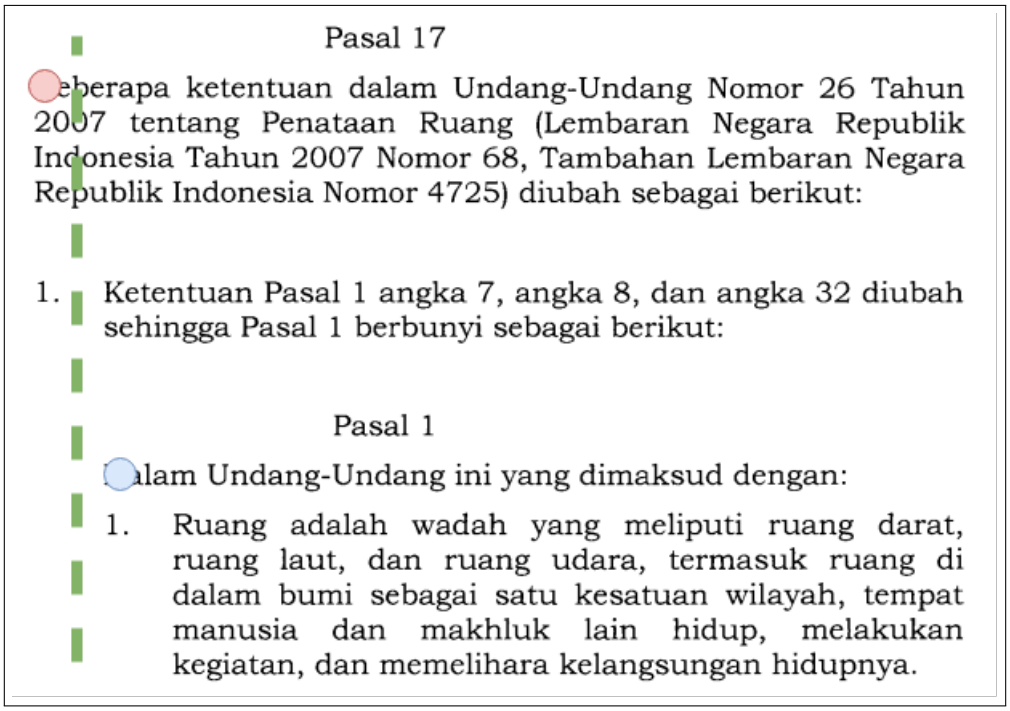
Pasal 1

Dalam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

Gambar 4.5: Contoh Amendemen pada UU 11/2020

Untuk menyelesaikan masalah ini, penulis menggunakan koordinat xL *span* tepat setelah *span* yang bertuliskan "Pasal X". Pada contoh deteksi kompoonen amendemen Gambar **??**, xL dari teks setelah pasal peng-amendemen (pasal biasa) ditandai dengan lingkaran merah, xL dari pasal yang diamendemen ditandai dengan lingkaran biru, dan garis hijau merepresentasikan batas pemeda xL antara pasal biasa dan pasal yang diamendemen. Pembedaan pasal biasa dan pasal amendemen hanya dilakukan jika dokumen memiliki komponen berupa amendemen.



Pasal 17

Beberapa ketentuan dalam Undang-Undang Nomor 26 Tahun 2007 tentang Penataan Ruang (Lembaran Negara Republik Indonesia Tahun 2007 Nomor 68, Tambahan Lembaran Negara Republik Indonesia Nomor 4725) diubah sebagai berikut:

1. Ketentuan Pasal 1 angka 7, angka 8, dan angka 32 diubah sehingga Pasal 1 berbunyi sebagai berikut:

Pasal 1

Dalam Undang-Undang ini yang dimaksud dengan:

1. Ruang adalah wadah yang meliputi ruang darat, ruang laut, dan ruang udara, termasuk ruang di dalam bumi sebagai satu kesatuan wilayah, tempat manusia dan makhluk lain hidup, melakukan kegiatan, dan memelihara kelangsungan hidupnya.

Gambar 4.6: Deteksi Komponen amendemen

4.7 Deteksi Rujukan

Setelah semua *span* dikelompokan menjadi suatu komponen, dilakukan deteksi rujukan pada teks pada setiap komponen untuk mengetahui apakah komponen tersebut menyebut komponen atau dokumen peraturan perundang-undangan lainnya. Tabel **??** adalah daftar pola rujukan yang berhasil dideteksi pada penelitian ini. Selain URI, data rujukan juga mengandung informasi seperti yang dijelaskan pada Tabel **??**. Contoh teks dan data rujukan yang terdeteksi dari teks tersebut dapat dilihat pada Kode **??**.

No.		
1	Pola	UUD 1945
	Regex	`Undang Undang Dasar Negara Republik Indonesia Tahun 1945`
	Contoh teks	Undang Undang Dasar Negara Republik Indonesia Tahun 1945
	URI	`:uud`
2	Pola	Undang-Undang Nomor {x} Tahun {y}
	Regex	`Undang-Undang Nomor [0-9]+ Tahun [0-9]+`
	Contoh teks	Undang-Undang Nomor 26 Tahun 2007
	URI	`:uu/2007/26`
3	Pola	Pasal {x}
	Regex	`Pasal [0-9]+`
	Contoh teks	Pasal 156
	URI	`:uu/2003/13/pasal/156`
4	Pola	ayat ({x})
	Regex	`ayat \((l|[0-9]+)\)`
	Contoh teks	ayat (1)
	URI	`:uu/2003/13/pasal/169/ayat/1`
4	Pola	Pasal {x} ayat ({x})
	Regex	`Pasal [0-9]+ ayat \((l|[0-9]+)\)`
	Contoh teks	Pasal 156 ayat (2)
	URI	`:uu/2003/13/pasal/156/ayat/2`
4	Pola	huruf {x} dan huruf {y}
	Regex	`huruf ?([a-z]?,? ?)+( [a-z]( |,))`
	Contoh teks	huruf a dan b
	URI	`uu/2003/13/pasal/1/point/5/point/a`
		`uu/2003/13/pasal/1/point/5/point/b`
4	Pola	huruf {x}, {y}, ... dan {z}
	Regex	`huruf ?([a-z]?,? ?)+( [a-z]( |,))`
	Contoh teks	huruf a, b, c, d, dan e
	URI	`uu/2003/13/menimbang/point/a`
		`uu/2003/13/menimbang/point/b`
		`uu/2003/13/menimbang/point/c`
		`uu/2003/13/menimbang/point/d`
		`uu/2003/13/menimbang/point/e`

Tabel 4.3: Pola, contoh teks, dan contoh URI deteksi rujukan

Nama	Deskripsi
`start`	*index* karakter awal rujukan
`end`	*index* karakter akhir rujukan
`uri`	URI dokumen dan

Tabel 4.4: Data rujukan

```yaml
textString: >-
  Pekerja/buruh yang diputus hubungan kerjanya berdasarkan alasan
  sebagaimana dimaksud dalam ayat (1), dapat memperoleh uang
  penggantian hak sebagaimana dimaksud dalam Pasal 156 ayat (4).
references:
  - start: 91
    end: 99
    uri: /uu/2003/13/pasal/158/ayat/1
  - start: 166
    end: 184
    uri: /uu/2003/13/pasal/156/ayat/4
```

Kode 4.6: Contoh teks pada Pasal 158 Ayat (4) UU 13/2003 beserta rujukan yang terdeteksi

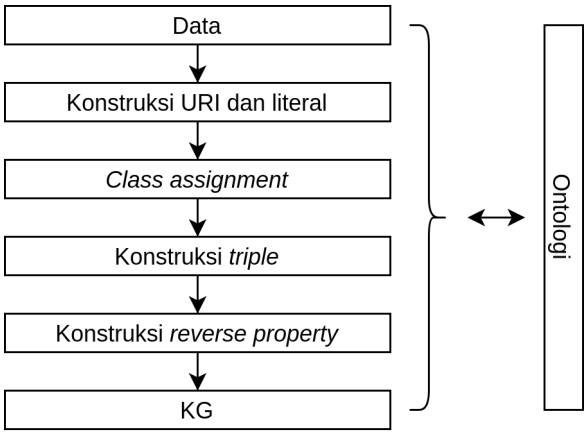
Untuk melakukan deteksi, teks akan melalui fungsi yang masing-masing memiliki pola untuk dideteksi. Oleh karena itu, untuk *substring* yang beririsan dapat memiliki lebih dari dua rujukan yang terdeteksi. Pada kasus tersebut akan diambil rujukan dengan *substring* terpanjang. Sebagai contoh, teks "Pasal 156 Ayat (4)" pada UU No.13 Tahun 2003 Pasal 158 Ayat (4) akan terdeteksi sebagai 3 URI rujukan seperti yang terlihat pada Tabel **??**.

Teks	URI
Pasal 156 Ayat (4)	`/uu/2003/13/pasal/156/ayat/4`
Pasal 156	`/uu/2003/13/pasal/156`
Ayat (4)	`/uu/2003/13/pasal/158/ayat/4`

Tabel 4.5: Contoh teks dan URI yang terdeteksi dari teks tersebut

Pada kasus ini, yang akan diambil adalah teks "Pasal 156 ayat (4)" karena merupakan teks terpanjang.

4.8 Konstruksi KG



Gambar 4.7: Konstruksi KG

Seluruh proses konstruksi KG didasari dengan ontologi yang sudah dirancang pada bab sebelumnya. KG dikonstruksi dari data peraturan perundang-undangan yang sudah terdapat sebagai data terstruktur dalam memori. Setiap objek seperti peraturan dan komponen akan dibangun menjadi URI berdasarkan Tabel **??** dan Tabel **??**. Literal seperti bilangan, *string*, dan tanggal juga dikonversi sesuai spesifikasi *literal type* pada Turtle. Setiap objek juga akan di-*assign* ke masing-masing *class* sesuai yang terdapat pada Tabel **??**. Kemudian *triple* dibangun berdasarkan relasi antara peraturan atau komponen dengan properti, domain, dan range sesuai spesifikasi pada Tabel **??**. Untuk setiap properti struktur, akan dibuat *reverse property* nya yaitu `o:bagianDari`. Sebagai contoh, jika terdapat *triple* `uu:2020/11/bab o:bab uu:2020/11/bab/1`, *reverse triple* nya adalah `uu:2020/11/bab/1 o:bab uu:2020/11/bab`. Properti struktur antaralain adalah `o:ayat`, `o:bab`, `o:bagian`, `o:daftarAyat`, `o:daftarBab`, `o:daftarBagian`, `o:daftarHuruf`, `o:daftarParagraf`, `o:daftarPasal`, `o:huruf`, `o:paragraf`, `o:pasal`, `o:segmen`, `o:teks`, dan `o:versi`.

4.9 Query

Query dilakukan terhadap KG untuk evaluasi *competency question* dan membuat demonstrasi untuk *use case evaluation*. Apache Jena Fuseki[3] digunakan sebagai SPARQL server yang menyimpan KG dan melakukan *query* terhadap KG. Di dalam folder Fuseki, Fuseki dapat dijalankan dengan KG yang memuat berkas Turtle `lex2kg.ttl` dan memiliki endpoint SPARQL */lex2kg* dengan Kode **??**. Setelah command tersebut dijalankan, SPARQL *query* dapat dilakukan menggunakan *client library* dengan endpo-

[3] `https://jena.apache.org/documentation/fuseki2/`

int http://127.0.0.1:3030/lex2kg/sparql, atau pada antarmuka yang tersedia pada http://127.0.0.1:3030/dataset.html?tab=query&ds=/lex2kg. Pada penelitian ini digunakan *client library* fetch-sparql-endpoint[4] untuk melakukan *query* dari bahasa pemrograman typescript.

```
java -jar fuseki-server.jar --file=lex2kg.ttl /lex2kg
```

Kode 4.7: Command untuk menjalankan Fuseki

4.10 Konversi Skala Besar

Penulis melakukan konversi skala besar pada PDF UU yang terdapat pada Laman Undang-Undang Jaringan Dokumentasi dan Informasi Hukum DPR RI.[5] Dari 1669 berkas PDF yang diunduh, 784 diantaranya berhasil dikonversi menjadi KG dengan total 1.163.051 *triple*. Jumlah tripel dikelompokan berdasarkan jenis informasi dapat dilihat pada Tabel **??**. Semua berkas PDF, berkas Turtle hasil konversi, dan berkas *artifact* seperti PDF hasil OCR, dan *span* dari konversi skala besar dapat diakses pada repositori https://github.com/aabccd021/uuri.

Jenis Informasi	Jumlah Triple
metadata peraturan	6.689
struktur peraturan	714.888
teks	152.301
rujukan dan amendemen	32.577
class assignment	256.416

Tabel 4.6: Analisis *triple* KG hasil konversi skala besar

[4]https://www.npmjs.com/package/fetch-sparql-endpoint
[5]https://www.dpr.go.id/jdih/uu

BAB 5
EVALUASI

Bab ini berisi evaluasi *use case* KG yang dihasilkan oleh Lex2KG. Evaluasi yang akan dilakukan adalah berupa SPARQL *query chat bot* sederhana, dan visualisasi peraturan perundang-undangan.

5.1 SPARQL Query

Dengan menggunakan SPARQL server seperi Apache Jena Fuseki, KG yang dibuat dengan Lex2KG dapat dianalisis dengan menjalankan SPARQL *query*. SPARQL *query* memiliki fitur operasi *join*, *union*, *optional*, dan *negation pattern*. SPARQL juga menyediakan *query* analisis yang lengkap seperti *filter*, *aggregate*, dan *property path*. Pada subbab ini akan ditujukan *competency question* beserta SPARQL *query* untuk menjawab pertanyaan tersebut.

5.1.1 *Competency question* No. 10

Competency question No. 10 "Tampilkan banyaknya penyisipan, pengubahan, dan penghapusan pasal yang dilakukan UU Cipta Kerja terhadap UU lain." dapat dijawab dengan melakukan *query* Kode **??** yang akan memberikan output Tabel **??**. Pada *query* digunakan properti `o:menghapus`, `o:menyisipkan`, dan `o:mengubah` untuk mencari *triple* yang melakukan amendemen.

```
PREFIX o: <https://example.org/lex2kg/ontology/>

SELECT ?type (COUNT(*) AS ?jumlah) WHERE {
  {
    ?point o:bagianDari+ <https://example.org/lex2kg/uu/2020/11> .
    ?point o:menghapus ?pasal .
    BIND("menghapus" AS ?type)
  } UNION {
    ?point o:bagianDari+ <https://example.org/lex2kg/uu/2020/11> .
    ?point o:menyisipkan ?pasal .
    BIND("menyisipkan" AS ?type)
  } UNION {
    ?point o:bagianDari+ <https://example.org/lex2kg/uu/2020/11> .
    ?point o:mengubah ?pasal .
    BIND("mengubah" AS ?type)
  }
} GROUP BY ?type
LIMIT 3
```

Kode 5.1: SPARQL *query* untuk *competency question* No. 10

?type	?jumlah
"menyisipkan"	107
"menghapus"	184
"mengubah"	930

Tabel 5.1: Output *query* Kode **??**

5.1.2 *Competency question* No. 15

Competency question No. 15 "Tampilkan semua peraturan serta peraturan yang ditimbangnya." dapat dijawab dengan melakukan *query* Kode **??** yang akan memberikan output Tabel **??**. Pada *query* digunakan `o:menimbang` untuk mencari entitas "menimbang", `o:bagianDari*` untuk mendapatkan komponen dari entitas "menimbang".

```
ocrmypdf -l ind --force-ocr --jobs 4 --tesseract-config tesseract-config.cfg UU-2021-
```

Kode 5.2: SPARQL *query* untuk *competency question* No. 15

?penimbang	?ditimbang
uu:2020/8	uu:2018/12
uu:2020/6	uu:2020/2
uu:2020/6	uu:2015/1
uu:2020/6	uu:2014/1
uu:2020/2	uu:2020/1

Tabel 5.2: Output *query* Kode **??**

5.1.3 *Competency question* No. 20

Competency question No. 20 "Tampilkan semua peraturan yang melakukan amendemen dan banyaknya pasal yang amendemen oleh pasal tersebut." dapat dijawab dengan melakukan *query* Kode **??** yang akan memberikan output Tabel **??**.

```
- type: point
  key: 8
  text: >-
    Informasi ketenagakerjaan adalah gabungan, rangkaian, dan analisis data yang
    berbentuk angka yang telah diolah, naskah dan dokumen yang mempunyai arti,
    nilai dan makna tertentu mengenai ketenagakerjaan.
- type: point
  key: 9
  text: >-
    Pelatihan kerja adalah keseluruhan kegiatan untuk memberi, memperoleh,
    meningkatkan, serta mengembangkan kompetensi kerja, produktivitas, disiplin,
    sikap, dan etos kerja pada tingkat keterampilan dan keahlian tertentu sesuai
    dengan jenjang dan kualifikasi jabatan atau pekerjaan.
```

Kode 5.3: SPARQL *query* untuk *competency question* No. 20

?doc	?amandementCount
uu:2020/11	1221
uu:2011/8	20
uu:2009/26	11

Tabel 5.3: Output *query* Kode **??**

5.2 Visualisasi KG

Visualisasi data dapat meningkatkan visibilitas dan *explainability* KG. Pada KG dapat dilakukan visualisasi *node* dan *relation* antara *node*, atau visualisasi statistik dari KG tersebut. Pada Subbab ini akan diberikan beberapa contoh visualisasi menggunakan *library* visualisasi VizKG.[1]

5.2.1 *Word Cloud* UU Ketenagakerjaan

Kode **??** merupakan kode untuk melakukan visualisasi *word cloud* dari UU 13/2003 tentang Ketenagakerjaan. Teks yang digunakan untuk *word cloud* diambil dari teks-teks yang terdapat pada UU 13/2003. Hasil visualisasi pada Gambar **??** menunjukan bahwa UU ini paling banyak membicarakan tentang pekerja buruh dan perjanjian kerja.

```
sparql_query = """
PREFIX o: <https://example.org/lex2kg/ontology/>
SELECT* WHERE {
?komponen o:bagianDari* <https://example.org/lex2kg/uu/2003/13> ;
          a o:Segmen ;
          o:teks ?teks .
}
"""
words = vkg(sparql_query=sparql_query,
 ↪  sparql_service_url=sparql_service_url, chart='WordCloud')
words.plot()
```

Kode 5.4: Visualisasi "Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya" menggunakan VizKG

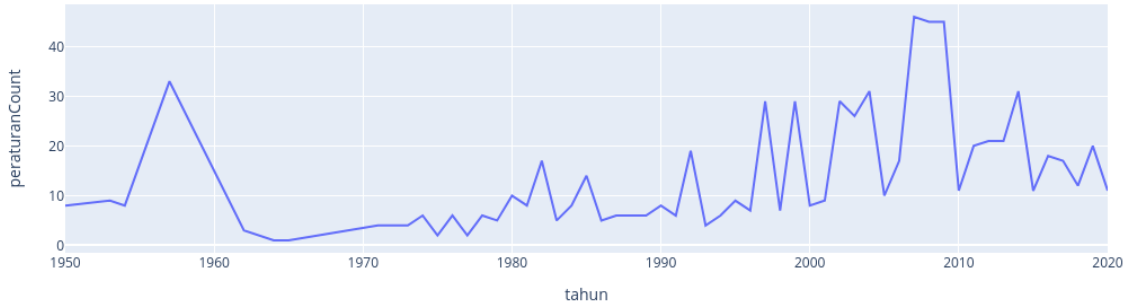
[1] https://pypi.org/project/VizKG/

Gambar 5.1: Output VizKG untuk Kode **??**

5.2.2 Diagram Garis Jumlah UU per Tahun

Kode **??** merupakan kode untuk melakukan visualisasi diagram garis jumlah undang-undang setiap tahunnya. Hasil visualisasi pada Gambar **??** menunjukan jumlah UU setiap tahunnya cenderung bertambah. Perlu diperhatikan bahwa UU yang terdapat pada KG hanya UU yang berhasil dikonversi oleh Lex2KG, oleh sebab itu kecenderungan penambahan jumlah UU bisa jadi disebabkan oleh kualitas PDF yang makin baik setiap tahunnya.

Kode 5.5: Visualisasi "Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya" menggunakan VizKG



Gambar 5.2: Output VizKG untuk Kode **??**

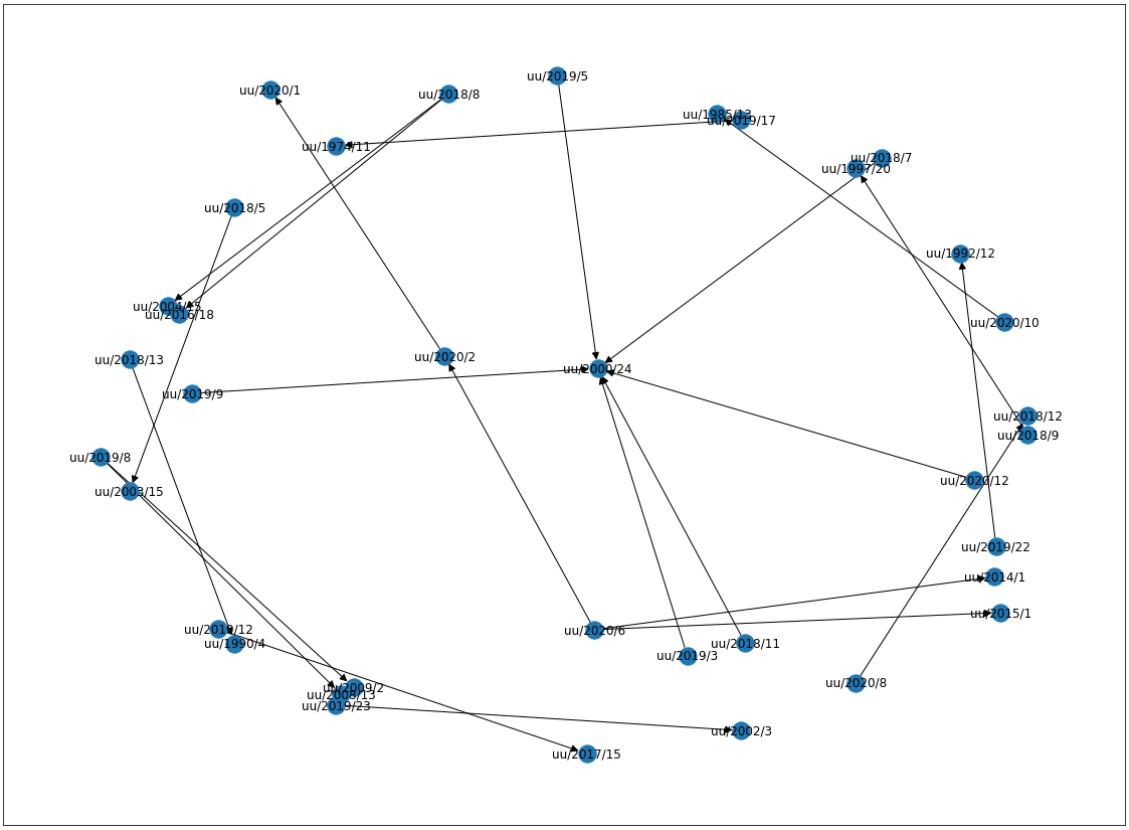
5.2.3 *Graph* "menimbang"

Kode **??** merupakan kode untuk melakukan visualisasi *graph* yang menunjukan hubungan `o:menimbang` antara peraturan, di mana peraturan penimbang disahkan setelah tahun 2017. Hasil visualisasi pada Gambar **??** bahwa tidak jarang sebuah peraturan ditimbang oleh atau menimbang lebih dari satu peraturan. Peraturan yang paling banyak ditimbang oleh UU setelah tahun 2017 adalah UU 24/2000 tentang Perjanjian Internasional.

```
sparql_query = """
PREFIX o: <https://example.org/lex2kg/ontology/>
PREFIX uu: <https://example.org/lex2kg/uu/>

SELECT DISTINCT ?penimbang ?penimbangStr ?ditimbang ?ditimbangStr
WHERE {
  ?penimbang o:menimbang ?menimbang .
  ?menimbangText o:bagianDari* ?menimbang .
  ?menimbangText o:merujuk ?ditimbang .
  ?ditimbang a o:Peraturan .
  ?penimbang o:tahun ?tahun .
  BIND(REPLACE(STR(?penimbang),"https://example.org/lex2kg/","") as
 ↪  ?penimbangStr) .
  BIND(REPLACE(STR(?ditimbang),"https://example.org/lex2kg/","") as
 ↪  ?ditimbangStr) .
  FILTER(?tahun > 2017)
}
"""
graph = vkg(sparql_query=sparql_query,
 ↪  sparql_service_url=sparql_service_url, chart="graph")
graph.plot()
```

Kode 5.6: Visualisasi "Tampilkan 5 peraturan dengan komponen terbanyak beserta jumlah komponennya" menggunakan VizKG



Gambar 5.3: Output VizKG untuk Kode **??**

5.3 *Chat bot* sederhana

KG peraturan perundang-undangan dapat digunakan pada *chat bot*. *Chat bot* sederhana memberikan pertanyaan yang sudah disediakan dan mengembalikan jawaban berdasarkan input pengguna sesuai program deterministik yang diimplementasi. Pada bab ini akan diperlihatkan bagaimana pengguna melalui beberapa skenario dan bagaimana *chat bot* nya bekerja. Skenario *chatbot* adalah sekuensial, artinya Skenario 2 adalah lanjutan dari Skenario 1, dan Skenario 3 adalah lanjutan dari Skenario 2. Input pengguna akan digunakan sebagai input dari template SPARQL *query* yang sudah tersedia. Teks yang ditampilkan *chat bot* ditandai dengan warna hitam, dan teks input pengguna ditandai dengan warna merah.

5.3.1 Skenario 1: Mencari Peraturan dengan Kata Kunci

Kode **??** merupakan tampilan chatbot pada Skenario 1 di mana pengguna dapat mencari peraturan perundang-undangan dengan kata kunci tertentu. Pada contoh ini pengguna mencari peraturan dengan kata kunci “kerja”, kemudian *chat bot* melakukan *query* terhadap KG menggunakan template Kode **??**, di mana `inputStr` memiliki nilai “kerja”. Ke-

mudian *chat bot* mengembalikan hasil *query* berupa nama peraturan perundang-undangan yang pada judulnya terdapa *substring* kerja.

```
Masukkan Kata Kunci Peraturan: kerja

Berikut adalah peraturan yang judulnya mengandung kerja.
Pilih salah satu untuk melihat lebih lanjut.
uu/2004/17
uu/2016/3
uu/1997/25
uu/2016/15
uu/2007/47
uu/1991/1
uu/2019/3

Pilih salah satu untuk melihat lebih lanjut:
```

Kode 5.7: Tampilan *chat bot* untuk Skenario 1

```
    const res = await sparqlQuery({
      queryStr: `
PREFIX o: <https://example.org/lex2kg/ontology/>
SELECT ?doc ?title
WHERE {
  ?doc a o:Peraturan .
  ?doc o:tentang ?title.
  FILTER REGEX(STR(?title), "${inputStr.toUpperCase()}")
}
`,
    });
```

Kode 5.8: *Query* template untuk Skenario 1

5.3.2 Skenario 2: Menampilkan Metadata Peraturan

Kode **??** merupakan tampilan chatbot pada Skenario 2 di mana pengguna memilih peraturan berdasarkan pilihan yang diberikan sebelumnya, kemudian menampilkan metadatanya.

Pada contoh ini pengguna memilih `uu/2019/3`, kemudian *chat bot* melakukan *query* terhadap KG menggunakan template Kode **??**, di mana `legalURI` memiliki nilai `uu/2019/3`. Kemudian *chat bot* mengembalikan hasil *query* berupa metadata UU 3/2019.

```
Pilih salah satu untuk melihat lebih lanjut: uu/2019/3

Data uu/2019/3 :
-metadata
-menimbang

Pilih data yang ingin anda lihat: metadata

yurisdiksi: Indonesia
jenisPeraturan: UU
tahun: 2019
bahasa: id
tentang: PENGESAHAN NOTA KESEPAHAMAN ANTARA PEMERINTAH REPUBLIK...
disahkanPada: 2009-01-10
disahkanDi: Jakarta
disahkanOleh: JOKO WIDODO
jabatanPengesah: PRESIDEN REPUBLIK INDONESIA,

Pilih data yang ingin anda lihat:
```

Kode 5.9: Tampilan *chat bot* untuk Skenario 2

```
    const res = await sparqlQuery({
      queryStr: `
PREFIX o: <https://example.org/lex2kg/ontology/>
SELECT ?label ?data
WHERE {
<https://example.org/lex2kg/${legalURI}> o:yurisdiksi|
 ↪  o:jenisPeraturan| o:tahun| o:bahasa| o:tentang| o:disahkanPada|
 ↪  o:disahkanDi| o:disahkanOleh| o:jabatanPengesah ?data ;
    ?label ?data .
}
PREFIX o: <https://example.org/lex2kg/ontology/>
`,
    });
```

Kode 5.10: *Query* template untuk Skenario 2

5.3.3 Skenario 3: Melihat Peraturan yang ditimbang oleh Peraturan

Kode **??** merupakan tampilan chatbot pada Skenario 3 di mana pengguna memilih peraturan berdasarkan pilihan yang diberikan sebelumnya, kemudian menampilkan peraturan yang ditimbangnya. Pada contoh ini pengguna memilih `uu/2019/3`, kemudian *chat bot* melakukan *query* terhadap KG menggunakan template Kode **??**, di mana `legalURI` memiliki nilai `uu/2019/3`. Kemudian *chat bot* mengembalikan hasil *query* berupa dokumen yang ditimbang UU 3/2019 yaitu UU 24/2000.

```
Pilih salah satu untuk melihat lebih lanjut: uu/2019/3

Data uu/2019/3 :
-metadata
-menimbang

Pilih data yang ingin anda lihat: menimbang

uu/2000/24
```

Kode 5.11: Tampilan *chat bot* untuk Skenario 3

```
    const res = await sparqlQuery({
      queryStr: `
PREFIX o: <https://example.org/lex2kg/ontology/>
SELECT DISTINCT ?ditimbang
WHERE {
  <https://example.org/lex2kg/${legalURI}> o:menimbang ?menimbang .
  ?menimbangText o:bagianDari* ?menimbang .
  ?menimbangText o:merujuk ?ditimbang .
}
`,
    });
```

Kode 5.12: *Query* template untuk Skenario 3

BAB 6
PENUTUP

Pada bab ini, Penulis akan memaparkan kesimpulan penelitian dan saran untuk penelitian berikutnya.

6.1 Kesimpulan

Berikut ini adalah kesimpulan terkait pekerjaan yang dilakukan dalam penelitian ini:

1. **Poin pertama**
   Penjelasan poin pertama.
2. **Poin kedua**
   Penjelasan poin kedua.

Tulis kalimat penutup di sini.

6.2 Saran

Berdasarkan hasil penelitian ini, berikut ini adalah saran untuk pengembangan penelitian berikutnya:

1. Saran 1.
2. Saran 2.

LAMPIRAN

LAMPIRAN 1: CHANGELOG

lorem ipsum